

UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Laurea Triennale in Informatica

TESI DI LAUREA

Titolo della Tesi

RELATORE

Prof. Fabio Palomba

Dott. Nome tutor

Università degli Studi di Salerno

CANDIDATO

Marco Donatiello

Matricola: 0512119045

Anno Accademico 2025-2026

Questa tesi è stata realizzata nel



Dedica o citazione

Abstract

Il testo dell'abstract qui...

Indice

Elenco delle Figure **iv**

Elenco delle Tabelle **v**

Abbreviazioni e glossario **vi**

1 Introduzione **1**

2 Analisi dello stato del arte e contesto tecnologico **2**

2.1 OWASP Cloud Security Top 10 2

2.1.1 IaC misconfiguration 4

2.2 API Security Risks 6

2.2.1 API1:2023 – Broken Object Level Authorization 8

2.2.2 API2:2023 – Broken Authentication 14

2.2.3 API3:2023 – Broken Object Property Level Authorization . . . 15

2.2.4 API4:2023 – Unrestricted Resource Consumption 18

2.2.5 API5:2023 – Broken Function Level Authorization 19

2.2.6 API6:2023 – Unrestricted Access to Sensitive Business Flows . 21

2.2.7 API7:2023 – Server Side Request Forgery 23

2.2.8 API8:2023 – Security Misconfiguration 24

2.2.9 API9:2023 – Improper Inventory Management 26

2.2.10 API10:2023 – Unsafe Consumption of APIs 28
2.2.11 Sintesi: dove si trova il difetto 29

3 Metodo di ricerca **31**
3.1 Approccio metodologico 31
3.1.1 Il criterio di classificazione 32
3.1.2 Requisiti derivati per lo strumento di rilevazione 33
3.2 Architettura del sistema 34
3.3 Rilevamento delle IaC misconfiguration 35
3.3.1 Lo scanner Checkov e la traduzione nel modello di dominio . 35
3.3.2 Correlazione e punteggio di rischio 37
3.4 Categorie ibride: analisi statica e residuo dinamico 38
3.4.1 API1 – Il legame fra oggetto e chiamante 40
3.4.2 API2 – L'identità del chiamante e la validità dei token 43
3.4.3 API3 – Le proprietà protette di un oggetto 46
3.4.4 API5 – Il ruolo richiesto per invocare una funzione 48
3.5 Categorie tecniche: un metodo per categoria 50
3.5.1 API4 – I limiti sul consumo di risorse, su tre livelli 50
3.5.2 API7 – Dall'input dell'utente alla chiamata di rete 52
3.5.3 API8 – La configurazione confrontata con i valori di riferimento 54
3.5.4 API10 – La fiducia nelle risposte dei servizi esterni 55

4 Validazione preliminare dello strumento **58**
4.1 Obiettivi e impostazione della validazione 58
4.1.1 Domande di ricerca 58
4.1.2 Le evidenze raccolte 59
4.2 Risultati 60
4.2.1 Casi di prova 60
4.2.2 Codice non scritto per i moduli 60
4.2.3 Insieme di prova 62
4.3 Discussione per categoria 64
4.3.1 Categorie ibride 64
4.3.2 Categorie tecniche 64

4.3.3 Categorie senza modulo 65
4.4 Risposta alle domande di ricerca 65
4.5 Limiti 67

5 Conclusioni **68**

Bibliografia **69**

Elenco delle figure

3.1 Diagramma dei componenti del sistema 34

Elenco delle tabelle

3.1 Policy Checkov sullo storage 36
3.2 Regole statiche della BOLA 42
3.3 Batteria di prove dinamiche della Broken Authentication 45
3.4 Regole statiche della BOPLA 47
3.5 Regole del modulo Broken Function Level Authorization 49
3.6 Condizioni rilevate dal modulo Unrestricted Resource Consumption 51
3.7 Regole del modulo Server-Side Request Forgery 53
3.8 Regole del modulo Security Misconfiguration 55
3.9 Regole del modulo Unsafe Consumption of APIs 56

4.1 Esiti sulle coppie gemelle 61
4.2 Riferimento di verità di crAPI ed esiti dello strumento 62
4.3 Esiti sull'insieme di prova 63
4.4 Metriche di accuratezza sull'insieme di prova 63

Abbreviazioni e glossario

Abbreviazioni

ACL	Access Control List
API	Application Programming Interface
AWS	Amazon Web Services
BFLA	Broken Function Level Authorization
BOLA	Broken Object Level Authorization
BOPLA	Broken Object Property Level Authorization
CORS	Cross-Origin Resource Sharing
CVSS	Common Vulnerability Scoring System
CWE	Common Weakness Enumeration
DoS	Denial of Service
HSTS	HTTP Strict Transport Security
HTTP	Hypertext Transfer Protocol
IaC	Infrastructure as Code
IAM	Identity and Access Management
JSON	JavaScript Object Notation
JWKS	JSON Web Key Set
JWT	JSON Web Token
MFA	Multi-Factor Authentication
NIST	National Institute of Standards and Technology
OAuth	Open Authorization
OIDC	OpenID Connect
OTP	One-Time Password
OWASP	Open Worldwide Application Security Project
PKCE	Proof Key for Code Exchange
ReDoS	Regular expression Denial of Service
REST	Representational State Transfer
RQ	Research Question (domanda di ricerca)
S3	Amazon Simple Storage Service
SAML	Security Assertion Markup Language
SSO	Single Sign-On
SSRF	Server-Side Request Forgery
TLS	Transport Layer Security
URL	Uniform Resource Locator
UUID	Universally Unique Identifier
YAML	YAML Ain't Markup Language

Glossario

Adapter Componente che esegue uno strumento esterno e ne traduce l'uscita nel modello di dominio del sistema.

Allowlist e denylist Elenco dei valori ammessi e, rispettivamente, di quelli esclusi; una denylist lascia passare tutto ciò che non elenca.

Analisi della contaminazione (*taint analysis*) Tecnica che segue un dato controllato dall'esterno dal punto in cui entra nel programma (*sorgente*) fino all'operazione sensibile in cui viene usato (*sink*).

Analisi statica e dinamica La prima esamina codice, configurazioni e contratti senza eseguire il sistema; la seconda invia richieste al sistema in esecuzione e ne osserva le risposte.

Autenticazione e autorizzazione La prima accerta chi effettua la richiesta; la seconda stabilisce se quel soggetto possa compiere un'operazione su una risorsa.

Bucket Contenitore di oggetti in un servizio di storage cloud come Amazon S3, con permessi propri che lo rendono pubblico o privato.

CWE Catalogo pubblico delle debolezze software mantenuto da MITRE; ogni voce (CWE seguita da un numero) descrive una classe di difetto.

Decoratore Costrutto che avvolge una funzione per aggiungerle un comportamento; nei framework web protegge tipicamente una rotta, come `login_required`.

Endpoint Punto di accesso di un'API, identificato da un URL e da un metodo HTTP, che espone un'operazione su una risorsa.

Falso positivo e falso negativo Segnalazione che non corrisponde a un difetto reale e, rispettivamente, difetto reale non segnalato.

Finding Singola segnalazione di uno strumento, espressa nel modello comune del sistema con posizione, categoria, severità e confidenza.

Hardening (irrobustimento) Difesa aggiuntiva, come la cifratura o la registrazione degli accessi, la cui assenza non apre da sola un varco.

IaC e Terraform Con l'*Infrastructure as Code* le risorse cloud sono dichiarate in file versionati; Terraform è lo strumento che li traduce nelle risorse reali.

Iniezione (*injection*) Attacco in cui un dato esterno viene interpretato come comando, per esempio in una query SQL.

Manifest File che dichiara risorse infrastrutturali (Terraform) o le dipendenze di un progetto (`requirements.txt`, `package.json`).

Mass assignment Copia automatica dei campi inviati dal client sugli attributi di un oggetto, che permette di valorizzare anche quelli interni.

Middleware Componente che si interpone fra la richiesta e il suo handler, applicando funzioni comuni a più rotte come l'autenticazione.

OpenAPI Formato standard per descrivere un'API REST (endpoint, parametri, schemi, sicurezza); il documento è detto *contratto*.

Policy Documento che dichiara quali soggetti possono eseguire quali azioni su quali risorse, per esempio su un bucket.

Rate limiting Limite al numero di richieste che un client può inviare in un intervallo di tempo.

Recall e precisione Quota di difetti reali trovati dallo strumento e, rispettivamente, quota di segnalazioni che corrispondono a difetti reali.

Redirezione Risposta con cui un server rimanda il client a un altro indirizzo.

Riferimento di verità (*ground truth*) Elenco delle vulnerabilità effettivamente presenti in un bersaglio, con cui si confrontano le segnalazioni.

Rotta e handler La rotta associa un percorso e un metodo HTTP a una funzione, l'*handler*, che serve la richiesta.

Serializzazione Conversione di un oggetto in un formato trasmissibile come JSON; se automatica può esporre attributi interni.

Token Stringa rilasciata dopo l'autenticazione e inviata con ogni richiesta per dimostrare l'identità del chiamante; un JWT contiene affermazioni firmate (*claim*) come l'utente, il ruolo e la scadenza.

CAPITOLO 1

Introduzione

Alcune linee guida per scrivere la tesi: 1) Ogni immagine/tabella deve avere le seguenti caratteristiche: a) Una caption; b) Deve essere esplicitamente commentata all'interno del testo (ricordatevi che se vi riferite alla figura/tabella inserendo anche il corrispettivo numero e.g., Figura X mostra che... allora la prima lettera deve essere scritta in maiuscolo; E ricordatevi che latex mette a disposizione un comando per citare correttamente (ref) senza dover mettere a mano il numeretto corrispondente) c) Le immagini devono essere rifatte tramite PPT o similari e salvate in pdf prima di inserirle nel file latex, altrimenti rischiate di inserire immagini sgranate. 2) Ricordatevi sempre di citare le fonti e di rifrasare, altrimenti rischiate di inccorrere nel reato di plagio (che può comportare anche la revoca del titolo accademico) e che citare non sminuisce il vostro lavoro, anzi, lo rafforza. 3)L'Abstract deve necessariamente contenere le seuguenti informazioni a) Un breve riasssunto della problematica; b)Una breve descrizione di ciò che è stato fatto; c) I risultati ottenuti.

CAPITOLO 2

Analisi dello stato del arte e contesto tecnologico

2.1 OWASP Cloud Security Top 10

Il portale AppSecMaster ha elaborato una propria classificazione dei dieci rischi più comuni e di maggiore impatto nei moderni ambienti cloud, ispirandosi ai principi promossi da OWASP in materia di sicurezza applicativa, che vanno dall'infrastruttura mal configurata alla scarsa visibilità delle risorse in esecuzione. Di seguito è riportata la lista delle principali problematiche individuate:

1. **Security Misconfiguration** — Bucket di storage lasciati con permessi troppo permissivi o istanze di calcolo distribuite senza le dovute restrizioni restano tra le cause più frequenti di violazione dei dati in ambienti cloud.

2. **Identity / Access Mismanagement** — Ruoli IAM assegnati con privilegi superiori a quelli effettivamente necessari, e mai rivisti dopo la configurazione iniziale, espongono i sistemi a un concreto rischio di escalation dei permessi.

3. **Improper Secrets/Key Management** - Se chiavi API e credenziali vengono salvate in chiaro o incorporate nel codice sorgente, la loro compromissione può aprire un accesso laterale a più servizi collegati.

4. **Insecure Network Segmentation** — Senza una reale separazione tra reti e ambienti diversi, un attaccante che viola un singolo componente può muoversi liberamente verso risorse più sensibili.

5. **Injection / Input Validation Flaws in Cloud Services** — Le classiche vulnerabilità di injection assumono forme nuove nei contesti cloud-native, colpendo funzioni serverless, code di messaggi e gateway API dove i controlli sugli input non sono sempre applicati in modo uniforme.

6. **Insecure Deserialization / API Exposure** — La gestione non sicura di dati serializzati, unita ad API interne prive di adeguati controlli di autenticazione o versionamento, costituisce un punto d'ingresso sfruttabile per eseguire codice non autorizzato.

7. **Vulnerable Third-Party/Container Dependencies** — Librerie open source e immagini container non aggiornate accumulano vulnerabilità note, soprattutto quando le stesse basi vengono riutilizzate tra più progetti senza verifiche periodiche.

8. **Improper Asset Inventory / Visibility** — Risorse cloud dimenticate o mai tracciate correttamente, incluse istanze abbandonate e servizi "ombra", sfuggono facilmente al controllo dei team di sicurezza.

9. **Insufficient Logging / Monitoring** — Log distribuiti su decine di servizi diversi e privi di un formato coerente rendono difficile individuare per tempo un'intrusione in corso.

10. **Unvalidated Redirects/Cloud Function Abuse** — Funzioni serverless eseguite con permessi troppo ampi, combinate a controlli insufficienti sui redirect, possono essere sfruttate per aggirare le restrizioni di accesso e ottenere un punto d'appoggio nascosto nell'infrastruttura.

Tra le problematiche elencate, il presente lavoro si concentra sulla prima, indicata nel seguito come IaC misconfiguration, e sulla componente di esposizione delle API della sesta, Insecure Deserialization / API Exposure: sono le due voci più direttamente legate agli obiettivi del sistema sviluppato e quelle che ricorrono con maggiore

frequenza negli scenari di autorizzazione tipici delle architetture basate su API. La prima è approfondita nella sottosezione seguente; la seconda nella Sezione 2.2, dove la classificazione OWASP dedicata alle API la articola in dieci categorie distinte.

2.1.1 IaC misconfiguration

Una IaC misconfiguration nasce da un errore nella configurazione di un servizio, di un prodotto o di una risorsa infrastrutturale: le impostazioni di sicurezza predefinite non vengono irrobustite a sufficienza, e questo lascia scoperto un fianco che con poco sforzo potrebbe essere protetto. I numeri parlano chiaro: il documento ufficiale OWASP riporta che il 90% delle applicazioni testate è risultato vulnerabile ad almeno una forma di misconfiguration [1]. Nella maggior parte dei casi non c'è nulla di sofisticato dietro: è una dimenticanza, un'impostazione lasciata di default, un permesso che nessuno ha mai rivisto. Ed è proprio questo che la distingue da una vulnerabilità nel codice applicativo — qui il problema non sta nella logica del software, ma in come le risorse sono state predisposte.

Le forme più comuni in cui una misconfiguration si manifesta sono, in fondo, poche e ricorrenti. Un bucket di storage lasciato accessibile pubblicamente, quando avrebbe dovuto restare privato, è probabilmente la più diffusa: basta una policy impostata male, magari copiata da un altro progetto senza troppa attenzione. Un container che gira con privilegi di root, invece, espone il sistema a un rischio diverso ma altrettanto concreto — se l'attaccante riesce a comprometterlo, si ritrova con permessi ben oltre quelli che gli servirebbero per fare danni. C'è poi il caso dei template Infrastructure-as-Code scritti senza le dovute verifiche: un file Terraform o CloudFormation che definisce una risorsa con permessi troppo ampi si porta dietro lo stesso errore ogni volta che viene riutilizzato, replicandolo su ambienti diversi senza che nessuno se ne accorga.

Un caso reale: il data breach Capital One Il caso Capital One del luglio 2019 resta probabilmente l'esempio più citato di questa categoria di rischio. Un bucket Amazon S3 lasciato con permessi troppo ampi permise a un'attaccante di sottrarre oltre 100 milioni di record clienti, tra dati anagrafici, indirizzi e informazioni finanziarie

sensibili [2]. Nessuna falla nella piattaforma cloud era coinvolta: la responsabilità ricadeva interamente sulla configurazione impostata dal cliente. Non a caso AWS, commentando l'accaduto, ribadì che *"Amazon S3 is secure by default"* [3], scaricando di fatto la colpa sulla gestione dei permessi lato utente. Sul piano economico il conto fu salato: 80 milioni di dollari di multa dall'Office of the Comptroller of the Currency, più 190 milioni versati per chiudere una class action. Capital One dimostra bene quanto una semplice disattenzione in fase di configurazione possa costare quanto, se non più di, un attacco costruito su un exploit tecnicamente elaborato.

Episodi come questo spiegano perché, negli ultimi anni, le organizzazioni abbiano iniziato ad affidarsi sempre meno alla revisione manuale delle configurazioni e sempre più a strumenti automatizzati capaci di individuare questo tipo di errori prima che vengano sfruttati. Una configurazione insicura, del resto, raramente viene introdotta di proposito: emerge quasi sempre da un template riutilizzato senza controllo, da un default mai cambiato, da una revisione saltata per mancanza di tempo — fattori che un controllo manuale sistematico difficilmente riesce a intercettare su larga scala e con continuità.

Il documento ufficiale OWASP illustra questa categoria di rischio attraverso quattro scenari esemplificativi [1]. Il primo riguarda applicazioni di esempio dimenticate su un server di produzione, spesso accompagnate da account amministrativi con credenziali mai modificate rispetto ai valori predefiniti. Il secondo descrive un server su cui la funzionalità di directory listing resta abilitata, consentendo a un attaccante di risalire fino al codice sorgente tramite reverse engineering. Il terzo riguarda messaggi di errore troppo dettagliati restituiti dall'application server, capaci di esporre informazioni sull'infrastruttura sottostante o versioni di componenti vulnerabili. Il quarto, infine, riguarda permessi di condivisione predefiniti di un cloud service provider lasciati aperti ad altri utenti su Internet, con conseguente esposizione di dati sensibili archiviati nello storage cloud.

2.2 API Security Risks

Le API oggi fanno gran parte del lavoro che una volta faceva il sito web stesso: non restituiscono una pagina pensata per essere letta da una persona, ma dati grezzi destinati a un'altra app, a un altro servizio, a un sistema che poi decide cosa farne. Lo schema delle richieste è prevedibile, gli endpoint espongono operazioni precise su risorse precise. Per un attaccante questo significa meno lavoro di interpretazione: non deve capire un'interfaccia grafica, gli basta osservare come sono costruite le chiamate.

OWASP mantiene, accanto al Top 10 per le applicazioni web, una classificazione separata dedicata alle API. Alcune categorie compaiono in entrambe le liste — la misconfiguration, il broken access control — ma buona parte dei rischi specifici delle API riguarda un livello che il Top 10 generale non copre: l'autorizzazione su singoli oggetti, proprietà, funzioni. Un'applicazione web tradizionale filtra cosa mostrare all'utente; un'API deve filtrare cosa restituire nella risposta. La differenza sembra minima, ma è da lì che nascono molte delle vulnerabilità trattate in questa sezione.

Le dieci categorie che compongono l'edizione 2023 riflettono questa logica: non rischi generici, ma rischi legati al modo specifico in cui un'API espone dati e funzionalità. Di seguito la lista completa, in ordine di gravità secondo OWASP [4]:

1. **API1:2023 – Broken Object Level Authorization (BOLA)** — un endpoint accetta l'ID di una risorsa senza verificare che chi lo richiede abbia davvero il diritto di accedervi; basta cambiare un numero nell'URL per leggere dati altrui.

2. **API2:2023 – Broken Authentication** — meccanismi di autenticazione implementati male, token che non scadono mai, o password recuperabili senza controlli sufficienti.

3. **API3:2023 – Broken Object Property Level Authorization** — qui il problema non è accedere alla risorsa sbagliata, ma vedere o modificare campi che non dovrebbero essere esposti o scrivibili da quel particolare utente.

4. **API4:2023 – Unrestricted Resource Consumption** — assenza di limiti su chiamate, dimensione dei payload o uso di risorse computazionali, con conseguenze che vanno dal semplice rallentamento a un vero e proprio denial of service.

5. **API5:2023 – Broken Function Level Authorization** — un utente normale riesce a invocare funzioni riservate agli amministratori, semplicemente perché il controllo sui permessi manca a livello di endpoint o di metodo HTTP.

6. **API6:2023 – Unrestricted Access to Sensitive Business Flows** — la singola richiesta può essere anche legittima, ma se ripetuta migliaia di volte automatizza un abuso della logica di business (es. acquisto massivo di biglietti, creazione automatica di account).

7. **API7:2023 – Server Side Request Forgery (SSRF)** — l'API accetta un URL fornito dall'utente e lo interroga senza validarlo, permettendo di raggiungere risorse interne altrimenti non accessibili dall'esterno.

8. **API8:2023 – Security Misconfiguration** — header di sicurezza mancanti, CORS troppo permissivo, ambienti di debug ancora attivi in produzione: stessa famiglia di errori vista nel capitolo precedente, qui applicata allo strato applicativo.

9. **API9:2023 – Improper Inventory Management** — endpoint vecchi, versioni deprecate o ambienti di staging dimenticati online, mai rimossi né monitorati, restano comunque raggiungibili da chi li trova.

10. **API10:2023 – Unsafe Consumption of APIs** — la vulnerabilità non nasce nell'API stessa, ma nella fiducia cieca riposta nei dati restituiti da API di terze parti integrate nel sistema.

Tra le dieci categorie appena elencate la prima merita un'attenzione diversa dalle altre: non tanto per la posizione in classifica, quanto per ciò che rappresenta, cioè un controllo di accesso che manca proprio dove ci si aspetterebbe fosse più solido, sulla singola risorsa richiesta. È il tipo di errore che si insinua facilmente in un endpoint scritto in fretta e che, allo stesso tempo, può esporre porzioni estese della base dati di un sistema con una richiesta HTTP modificata di poco. La sottosezione seguente lo esamina nel dettaglio: come si manifesta, da quali cause nasce e con quali misure lo si previene.

2.2.1 API1:2023 – Broken Object Level Authorization

Definizione e collocazione La Broken Object Level Authorization (BOLA) si manifesta quando un'applicazione, pur autenticando correttamente l'utente che effettua una richiesta, non verifica se quello stesso utente abbia effettivamente il diritto di accedere all'oggetto specifico richiesto [5].

Un'interfaccia di programmazione applicativa espone tipicamente un insieme di risorse — utenti, ordini, documenti, transazioni — identificabili singolarmente attraverso un riferimento univoco. L'accesso a tali risorse presuppone due verifiche concettualmente distinte, spesso confuse nella pratica implementativa: l'autenticazione, che accerta l'identità del soggetto che effettua la richiesta, e l'autorizzazione, che stabilisce se quel soggetto, una volta identificato, abbia il diritto di operare su una specifica risorsa.

Un utente autenticato correttamente non è, per definizione, un utente autorizzato ad accedere a qualunque oggetto gestito dal sistema. Questa distinzione, apparentemente elementare, è proprio il punto in cui molte implementazioni falliscono: il possesso di credenziali valide viene talvolta considerato — implicitamente, nel disegno del codice — condizione sufficiente per l'accesso a una risorsa, quando in realtà rappresenta soltanto la premessa per una verifica ulteriore, relativa alla relazione specifica tra quell'utente e quell'oggetto.

Funzionamento delle API e ruolo degli identificatori di risorsa Le API REST, e in misura diversa quelle basate su GraphQL, espongono le risorse attraverso identificatori che compaiono nel percorso della richiesta, nei parametri di query, nel corpo del messaggio o negli header. Un endpoint come `GET /api/orders/482` restituisce, nella maggior parte delle implementazioni, l'oggetto ordine identificato dal valore 482, indipendentemente dal fatto che tale valore sia un intero progressivo, un UUID o un altro tipo di riferimento.

La gestione di una richiesta di questo tipo comporta, lato server, due passaggi distinti: la risoluzione dell'identificatore in un oggetto concreto, tramite interrogazione del database o di un servizio dati, e il controllo che l'utente autenticato abbia diritto di accesso a quell'oggetto specifico. Il primo passaggio è quasi sempre implementato

correttamente, poiché è indispensabile al funzionamento stesso dell'endpoint; il secondo, non avendo un effetto visibile quando manca, viene più facilmente trascurato o dato per scontato in fase di sviluppo.

Va osservato che l'endpoint, in questo scenario, non è di per sé vietato all'utente: la sua configurazione di ruoli e permessi lo autorizza a invocare quella funzione API. Questo distingue concettualmente BOLA da un'altra categoria della stessa famiglia, la Broken Function Level Authorization (BFLA), in cui è l'accesso alla funzione stessa a non essere autorizzato. In BOLA la funzione è legittima; il problema riguarda esclusivamente quale istanza dell'oggetto venga restituita o modificata.

Meccanismo dell'attacco Lo sfruttamento di una vulnerabilità BOLA non richiede, nella maggior parte dei casi, strumenti sofisticati né la compromissione di credenziali altrui. È sufficiente disporre di un account legittimo sul sistema bersaglio. Il percorso tipico dell'attacco segue una sequenza riconoscibile: l'utente accede al proprio account, richiede una propria risorsa attraverso l'interfaccia normale dell'applicazione e osserva, tramite gli strumenti di sviluppo del browser o un proxy di intercettazione, l'identificatore associato a quella risorsa nella richiesta sottostante.

A questo punto l'identificatore viene sostituito con un altro valore plausibile per lo stesso schema — un intero vicino, nel caso di sequenze numeriche, oppure un valore ottenuto per enumerazione più sistematica quando lo schema lo consente — e la richiesta viene reinviata con lo stesso token di autenticazione, valido e non modificato. Ad esempio:

```
GET /api/orders/483 HTTP/1.1
Host: api.example.com
Authorization: Bearer <token_utente_A>
```

dove 483 non è l'ordine appartenente al titolare del token, ma un identificatore vicino a quello osservato in precedenza durante il normale utilizzo dell'applicazione. La richiesta è, dal punto di vista del protocollo, del tutto regolare: cambia soltanto il valore del parametro. Che la risposta contenga davvero i dati di un altro utente dipende da un unico fattore, ripreso nel paragrafo seguente: se il server confronti o meno l'identità del chiamante con il proprietario effettivo dell'oggetto.

Esempio applicativo Si consideri la risorsa `/api/orders/{id}`, utilizzata per recuperare i dettagli di un ordine. Una richiesta legittima, effettuata dal proprietario dell'ordine 482, ha la forma:

```
GET /api/orders/482 HTTP/1.1
Host: api.example.com
Authorization: Bearer <token_utente_A>
```

e produce una risposta simile alla seguente:

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "order_id": 482,
  "customer_id": 1041,
  "items": ["..."],
  "shipping_address": "..."
}
```

La richiesta modificata introdotta nel paragrafo precedente, diretta all'ordine 483 e appartenente a `customer_id` 1078, viene gestita da un'implementazione vulnerabile allo stesso modo: l'identificatore viene estratto dal percorso, la tabella corrispondente viene interrogata, il risultato viene restituito. In nessun punto di questo flusso il valore di `customer_id` restituito viene confrontato con l'identità associata al token presente nell'header `Authorization`. La risposta, di conseguenza, è ancora un `200 OK`, questa volta con i dati di un ordine che non appartiene al chiamante.

Il punto critico non è la struttura della richiesta, identica nei due casi, ma l'assenza di un confronto esplicito fra utente autenticato e risorsa richiesta. Un'implementazione corretta dovrebbe, prima di restituire l'oggetto, verificare che il `customer_id` dell'ordine coincida con l'identità del chiamante — o che quest'ultimo disponga di un diritto di accesso esplicitamente concesso su quella risorsa — e restituire, in caso contrario, un errore di autorizzazione: tipicamente `403 Forbidden`, oppure `404 Not Found` quando si preferisce non rivelare l'esistenza della risorsa a chi non ne ha diritto.

Cause e meccanismi tecnici tipici L'assenza di un controllo di autorizzazione a livello di oggetto è la causa immediata di ogni istanza di BOLA, ma raramente costituisce una scelta progettuale deliberata. Più spesso deriva da alcune condizioni che meritano di essere analizzate singolarmente.

La prima riguarda la sovrapposizione concettuale tra autenticazione e autorizzazione nella fase di implementazione. Un endpoint protetto da un middleware di autenticazione, che verifica la validità del token e ne estrae l'identità, può apparire sicuro a chi lo sviluppa, inducendo a trascurare la necessità di un controllo ulteriore specifico per l'oggetto acceduto. Il middleware risponde alla domanda "chi sta chiamando?", non a "questo chiamante ha diritto su questo oggetto?".

Una seconda causa è la fiducia implicita nei parametri forniti dal client. Alcune implementazioni derivano l'identità del proprietario di una risorsa dallo stesso identificatore fornito nella richiesta, oppure presuppongono che, poiché l'interfaccia utente dell'applicazione non espone mai identificatori altrui, un client "onesto" non li invierebbe mai. Questa assunzione ignora che un client HTTP può essere costruito o modificato arbitrariamente da chi effettua la richiesta, indipendentemente dall'interfaccia grafica ufficiale.

Un terzo fattore è rappresentato da controlli di autorizzazione implementati in modo incompleto o incoerente tra i diversi endpoint di una stessa API. È relativamente comune riscontrare verifiche corrette sull'operazione di cancellazione di una risorsa e verifiche assenti sull'operazione di lettura o aggiornamento della medesima risorsa, poiché ciascun endpoint viene sviluppato e sottoposto a revisione in momenti diversi, con criteri non sempre uniformi. La crescita organica di un'API nel tempo, con l'aggiunta progressiva di nuovi endpoint da parte di sviluppatori diversi, tende ad amplificare questa disomogeneità.

Una causa più strutturale riguarda infine la gestione delle relazioni tra utenti e risorse nel modello dati. Quando tale relazione non è rappresentata in modo esplicito e interrogabile — ad esempio tramite una chiave esterna diretta tra la tabella degli utenti e quella delle risorse, o tramite un livello di autorizzazione centralizzato — ogni endpoint deve reimplementare autonomamente la logica di verifica, e questo aumenta la probabilità che qualche percorso venga trascurato.

Impatto L'impatto di una vulnerabilità BOLA dipende dalla natura della risorsa esposta, ma incide tipicamente su più proprietà di sicurezza contemporaneamente. Sul piano della riservatezza, l'accesso non autorizzato a un oggetto consente la lettura di dati che dovrebbero rimanere riservati al proprietario legittimo: informazioni personali, dettagli finanziari, contenuti sanitari, comunicazioni private, a seconda del dominio applicativo.

Quando l'endpoint vulnerabile consente anche operazioni di scrittura, l'impatto si estende all'integrità delle risorse: un attaccante può modificare dati appartenenti ad altri utenti, alterare lo stato di un ordine, cambiare le impostazioni di un account che non gli appartiene. Se l'operazione esposta è una cancellazione, la conseguenza diventa la perdita di disponibilità della risorsa per il legittimo proprietario.

Un aspetto che distingue BOLA da molte altre classi di vulnerabilità è la sua natura sistematicamente ripetibile: una volta individuato un endpoint privo del controllo appropriato, lo stesso schema di attacco può essere applicato a un numero elevato di identificatori, trasformando quella che sembra una singola falla puntuale in un canale di accesso a porzioni estese della base dati.

Il rischio assume connotati particolari negli ambienti multi-tenant e cloud, dove più clienti condividono la stessa infrastruttura applicativa e la separazione tra i rispettivi dati dipende interamente dalla correttezza della logica di autorizzazione, non da una separazione fisica delle risorse. In un simile contesto, una singola vulnerabilità BOLA non compromette soltanto i dati di un utente, ma può erodere il confine logico tra tenant distinti, con implicazioni che vanno oltre il singolo incidente e toccano la fiducia complessiva riposta nell'infrastruttura condivisa.

Mitigazione La prevenzione di BOLA richiede che ogni accesso a una risorsa identificata da un parametro fornito dal client sia accompagnato da una verifica esplicita del diritto dell'utente su quello specifico oggetto, eseguita interamente lato server. Nessuna informazione proveniente dal client, incluso l'identificatore stesso della risorsa, può essere considerata prova di autorizzazione: l'identificatore indica quale oggetto viene richiesto, non chi ha diritto di accedervi.

Un principio applicabile in modo trasversale è quello del *least privilege*, secondo cui un soggetto dovrebbe disporre soltanto dei permessi strettamente necessari

a svolgere le proprie funzioni, senza margini di accesso più ampi per comodità implementativa. Applicato al livello oggetto, questo si traduce nella necessità di associare esplicitamente ogni utente, o ruolo, alle risorse su cui ha effettivamente diritto di operare, evitando che l'accesso derivi da assunzioni implicite sulla struttura delle richieste.

Dal punto di vista architetturale, è preferibile centralizzare la logica di autorizzazione in un livello comune — un modulo, un middleware o un servizio dedicato — piuttosto che reimplementarla in modo indipendente in ciascun endpoint. Questa scelta non elimina la necessità di applicare il controllo in ogni punto di accesso, ma riduce la probabilità che errori o dimenticanze locali producano percorsi non protetti, e semplifica la verifica di coerenza tra le diverse funzioni esposte dall'API.

La coerenza dei controlli tra operazioni di lettura, scrittura e cancellazione sulla stessa risorsa merita infine attenzione specifica: la protezione di un'operazione non implica automaticamente la protezione delle altre, ed è in questa asimmetria che si annida buona parte delle istanze reali di BOLA.

Le sottosezioni seguenti trattano in forma più sintetica le restanti categorie della OWASP API Security Top 10, a partire da API2:2023.

2.2.2 API2:2023 – Broken Authentication

Definizione e collocazione Broken Authentication occupa la seconda posizione della classificazione OWASP e si colloca concettualmente a monte di BOLA: mentre quest'ultima riguarda il controllo sull'accesso a uno specifico dato o risorsa, Broken Authentication compromette la capacità stessa del sistema di stabilire con certezza l'identità di chi effettua la richiesta. Un fallimento in questa fase rende vano qualsiasi controllo successivo, poiché un meccanismo di autorizzazione perde ogni efficacia se l'identità su cui si fonda può essere falsificata o aggirata [6]. A differenza di altre categorie più circoscritte, questa voce raggruppa un insieme eterogeneo di debolezze nei processi di verifica dell'identità: dalla gestione poco attenta dei token di sessione all'assenza di vincoli sul numero di tentativi di accesso consentiti. Un esempio comune riguarda i messaggi di risposta del server: se il sistema indica esplicitamente quale dei due campi, nome utente o password, sia errato, fornisce involontariamente all'attaccante una conferma sull'esistenza di quell'account. Parallelamente, l'assenza di limitazioni sulla frequenza delle richieste lascia l'interfaccia di login esposta ad attacchi automatizzati volti a indovinare le credenziali su larga scala.

Cause e meccanismi tecnici tipici Spesso queste problematiche nascono dal tentativo di implementare soluzioni di autenticazione proprietarie o non standard. Come evidenziato dalla documentazione OWASP, strumenti come le API key o protocolli di autorizzazione come OAuth vengono talvolta impiegati impropriamente per identificare gli utenti in scenari per cui non sono stati progettati, creando falle strutturali [6]. Un ulteriore fattore critico è la mancanza di controlli sul volume dei tentativi di accesso. Endpoint di autenticazione e flussi di recupero password privi di rate limiting risultano vulnerabili ad attacchi in cui elenchi di credenziali già compromesse altrove vengono provati in massa, sfruttando la tendenza diffusa a riutilizzare le medesime password su più servizi [7]. Infine, la gestione dei token di sessione (come i JSON Web Token, o JWT) introduce ulteriori insidie: errori nella convalida della firma crittografica, chiavi di cifratura deboli, mancato controllo dei tempi di scadenza o assenza di meccanismi di revoca immediata permettono a un attaccante di forgiare token validi o di continuare a utilizzare credenziali che dovrebbero essere

già decadute [8].

Impatto L'impatto di Broken Authentication è per sua natura sistemico: assumendo il controllo dell'identità di un utente, l'attaccante ne eredita tutti i privilegi operativi senza che il backend possa distinguere le azioni legittime da quelle malevole [6]. Ciò può tradursi non solo nell'accesso a dati riservati, ma anche nella manomissione dei parametri di sicurezza dell'account (come la modifica dell'indirizzo di recupero), trasformando un'intrusione temporanea nella perdita definitiva del profilo.

Mitigazione La contromisura principale consiste nell'affidarsi a standard industriali consolidati e a librerie verificate, evitando soluzioni "fai-da-te" per la gestione di token e password [9]. A livello applicativo e infrastrutturale è indispensabile introdurre politiche di rate limiting dedicate e restrittive sugli endpoint di login, adottare l'autenticazione a più fattori (MFA) dove possibile e imporre una nuova conferma delle credenziali prima di eseguire modifiche critiche sul profilo utente [10].

2.2.3 API3:2023 – Broken Object Property Level Authorization

Definizione e collocazione La *Broken Object Property Level Authorization* (BOPLA) identifica una classe di vulnerabilità nella quale i controlli di autorizzazione applicati alle API non considerano adeguatamente le singole proprietà degli oggetti gestiti dall'applicazione. Il problema non riguarda quindi soltanto la possibilità per un utente di accedere a un determinato oggetto, ma anche quali attributi di quell'oggetto possa effettivamente leggere o modificare [4].

La distinzione rispetto alla *Broken Object Level Authorization* (BOLA) è rilevante. Nel caso di BOLA, il controllo di autorizzazione fallisce a livello dell'oggetto: un utente riesce, per esempio, ad accedere a una risorsa appartenente a un altro utente manipolando l'identificativo della risorsa. Nel caso di BOPLA, invece, l'utente può avere legittimamente accesso all'oggetto, ma ottenere o modificare proprietà che non rientrano nei suoi privilegi. L'oggetto, quindi, non costituisce necessariamente la risorsa a cui viene negato l'accesso; è una sua specifica proprietà a essere esposta o manipolabile senza un'adeguata autorizzazione.

Il rischio è particolarmente rilevante nelle API REST, dove è frequente che un endpoint restituisca direttamente la rappresentazione serializzata di un oggetto. Una struttura interna composta da numerosi attributi può così diventare, in assenza di una selezione esplicita, il corpo stesso della risposta inviata al client.

Cause e meccanismi tecnici tipici Una delle cause più comuni è la sovrapposizione tra il modello utilizzato internamente dall'applicazione e quello esposto attraverso l'API. Un oggetto memorizzato nel database può contenere, ad esempio, proprietà quali `id`, `name`, `email`, `role` e `internalStatus`. Non tutte queste proprietà hanno però la stessa finalità né devono necessariamente essere accessibili allo stesso insieme di utenti.

Un'implementazione che serializza automaticamente l'intero oggetto può produrre una risposta contenente anche attributi che il client non dovrebbe conoscere. In questo caso il problema non deriva necessariamente da un accesso non autorizzato all'oggetto, ma dal fatto che l'API non applica una politica distinta alle proprietà che lo compongono. L'interfaccia grafica potrebbe nascondere determinati campi, ma ciò non costituisce una misura di sicurezza: il contenuto della risposta HTTP rimane osservabile dal client e può essere analizzato direttamente.

Un secondo meccanismo riguarda le operazioni di scrittura. Framework e librerie possono consentire il *binding* automatico dei parametri ricevuti dal client verso gli attributi di un oggetto applicativo. Se il server accetta un payload come:

```
{
  "name": "Mario",
  "email": "mario@example.com"
}
```

un attaccante può provare ad aggiungere proprietà che non erano previste dal normale flusso applicativo:

```
{
  "name": "Mario",
  "email": "mario@example.com",
```

```
  "role": "admin"
}
```

Se il campo `role` viene accettato e aggiornato senza un controllo specifico, l'utente può modificare una proprietà interna che dovrebbe essere gestita esclusivamente dal sistema. Lo stesso principio può riguardare attributi quali `isAdmin`, `approved`, `price` o `ownerId`, a seconda del modello utilizzato dall'applicazione.

Il problema può quindi manifestarsi in due direzioni differenti: lettura non autorizzata di proprietà e modifica non autorizzata di proprietà. Nel primo caso l'attaccante osserva informazioni che non dovrebbe ricevere; nel secondo riesce a influenzare direttamente lo stato dell'applicazione attraverso attributi che non dovrebbero essere controllabili dal client.

Impatto L'impatto di una vulnerabilità BOPLA dipende principalmente dal tipo di proprietà coinvolta e dalle funzioni che la utilizzano. La semplice esposizione di un attributo può comportare una violazione della riservatezza, mentre la possibilità di modificarlo può compromettere l'integrità dei dati o il funzionamento dell'applicazione.

Nel caso di proprietà contenenti informazioni personali, finanziarie o relative all'organizzazione interna del sistema, l'esposizione può determinare una divulgazione di dati a soggetti non autorizzati. La possibilità di modificare determinate proprietà può invece portare alla perdita o alla corruzione dei dati. In situazioni più critiche, una proprietà utilizzata nei meccanismi di autorizzazione o nella gestione dello stato dell'account può essere sfruttata per ottenere un'escalation dei privilegi o contribuire alla compromissione dell'account. La gravità della vulnerabilità dipende quindi dal significato che quella proprietà assume all'interno dell'applicazione, più che dal meccanismo che ne consente l'accesso.

Mitigazione La mitigazione di BOPLA richiede che l'autorizzazione venga applicata non soltanto all'endpoint o all'oggetto, ma anche alle proprietà che possono essere restituite o modificate. La prima misura consiste quindi nel definire esplicitamente quali attributi ogni endpoint può esporre e quali possono essere accettati in input.

Per le risposte è preferibile evitare i meccanismi di serializzazione automatica, che restituiscono l'oggetto così com'è rappresentato internamente, e selezionare invece le proprietà necessarie alla singola operazione.

Per le operazioni di scrittura è necessario limitare esplicitamente gli attributi modificabili dal client. Una tecnica comune consiste nell'utilizzare una *whitelist* delle proprietà ammesse, evitando che il payload venga associato automaticamente a tutti gli attributi dell'oggetto.

2.2.4 API4:2023 – Unrestricted Resource Consumption

Ogni richiesta servita da un'API consuma risorse: banda, CPU, memoria, storage e, con frequenza crescente, servizi a pagamento di terze parti. API4:2023 raccoglie i casi in cui almeno uno dei limiti che dovrebbero governare quel consumo è assente o dimensionato male, fra cui i timeout di esecuzione, la memoria allocabile e il numero di processi [11]. Non è il difetto di una funzione specifica, ma una proprietà trasversale di qualunque endpoint: i limiti di consumo sono la difesa in profondità di altre categorie, e la loro assenza si intreccia con gli endpoint di autenticazione di API2, con l'automazione dei flussi di business di API6, con la copertura incompleta della superficie esposta di API9 e con le chiamate verso servizi esterni di API10.

La radice comune è un'asimmetria: produrre una richiesta costa al client quasi nulla, servirla può costare al server una quantità che cresce in funzione di parametri che il client controlla, e finché quella funzione di costo non ha un limite superiore l'API resta esposta. I meccanismi sono tre. Il volume puro, dove più insidiosa dell'assenza del limite è la sua presenza in una sola dimensione, perché un tetto per indirizzo IP cade contro un attacco distribuito e uno per endpoint non intercetta il batching. La moltiplicazione di operazioni dentro una richiesta sola, illustrata dallo scenario GraphQL in cui centinaia di mutazioni aggregate in un unico batch saturano la memoria aggirando sia il limite di frequenza sia quello sulla dimensione delle immagini [11, 12]. Infine il costo algoritmico della singola richiesta [13], con il ReDoS come istanza più diffusa, documentata nei server JavaScript dove il thread singolo rende una richiesta sufficiente a congelare il processo [14]. Vi si aggiunge la causa che l'edizione 2023 porta in primo piano, l'assenza di soglie di spesa verso

i fornitori esterni, dove la risorsa non è computazionale ma economica. Che limiti tanto semplici manchino dipende da due fatti: possono risiedere nel framework, nel proxy, nel gateway o presso il fornitore cloud, e quando ogni livello presume che se ne occupi un altro non se ne occupa nessuno; e la loro assenza non produce errori, sicché un endpoint privo di limiti supera ogni test funzionale.

L'esito immediato è il denial of service per esaurimento, che CWE-770 enumera come conseguenza principale sulla disponibilità [15], ma il punto di rottura è la logica applicativa anziché la banda: una sola richiesta ReDoS o un solo batch GraphQL bastano, e nessuna difesa perimetrale basata sul volume li riconosce come anomali.

Nella mitigazione nessun livello basta da solo: i limiti fisici su memoria, CPU e processi sono dichiarati alla piattaforma, mentre all'ingresso dei dati va fissata una dimensione massima per ogni parametro e payload, vincolo che il contratto OpenAPI permette di dichiarare esplicitamente. Il rate limiting, segnalato al client con il codice `429` [16, 17], va tarato per endpoint e affiancato da un limite sulle esecuzioni della singola operazione per utente, che il conteggio delle chiamate non cattura [11], e si irrigidisce sugli endpoint di autenticazione [6, 9]; NIST lo colloca nel gateway anziché nel singolo servizio [18], scelta corretta purché ogni host esposto stia dietro quel gateway. Per la famiglia algoritmica servono timeout e motori a tempo lineare [14], per GraphQL l'analisi statica del costo di una query prima di eseguirla [12], per la dimensione economica soglie di spesa o allarmi di fatturazione.

2.2.5 API5:2023 – Broken Function Level Authorization

Un'API espone operazioni, e non tutte sono destinate a tutti. Creare un invito con privilegi amministrativi, esportare l'elenco degli utenti, cancellare un account altrui sono funzioni riservate a un ruolo, e il controllo che verifica quel ruolo prima di eseguirle è l'autorizzazione a livello di funzione: API5:2023 raccoglie i casi in cui tale controllo manca o è aggirabile, e un utente ordinario riesce a invocare funzioni riservate [19]. La differenza rispetto ad API1:2023 è di livello, ed è l'unica che OWASP fissi in modo prescrittivo: nella BOLA l'accesso all'endpoint è previsto e la violazione avviene manipolando l'identificatore dell'oggetto, mentre se l'attaccante raggiunge una funzione cui non dovrebbe accedere il caso ricade sotto la BFLA [5]. Il riferimento

tassonomico è CWE-285 *Improper Authorization*, che MITRE sconsiglia però di usare per mappare vulnerabilità reali, rimandando alle voci figlie CWE-862 quando il controllo è assente e CWE-863 quando esiste ma decide male [20].

Il controllo funzionale manca perché non ha un luogo obbligato: OWASP osserva che l'autorizzazione su una funzione è gestita tramite configurazione oppure a livello di codice, e che scrivere controlli corretti è confuso, date le molte gerarchie di ruoli e gruppi delle applicazioni moderne [19]. Vi si somma il valore predefinito, perché una rotta aggiunta a un controller eredita il routing e l'autenticazione, non l'autorizzazione: nasce raggiungibile e tale resta. Sfruttarla non richiede manipolazioni, bastano chiamate legittime verso un endpoint cui non si dovrebbe accedere, e la struttura prevedibile delle API rende il tentativo facile, sostituendo `GET` con `PUT` oppure `users` con `admins`: la stessa regolarità che rende un'API documentabile la rende enumerabile. Lo scenario di riferimento è un'applicazione ad accesso su invito, in cui l'app mobile recupera un invito già emesso con una chiamata del tutto ordinaria:

```
GET /api/invites/{invite_guid} HTTP/1.1
Host: api.example.com
Authorization: Bearer <token_utente_normale>
```

La risposta espone, fra gli altri campi, il ruolo associato all'invito. L'attaccante ne cambia allora metodo ed endpoint, riutilizzando lo stesso token e senza manipolare alcun identificatore:

```
POST /api/invites/new HTTP/1.1
Host: api.example.com
Authorization: Bearer <token_utente_normale>
Content-Type: application/json

{"email": "attaccante@example.com", "role": "admin"}
```

L'operazione è riservata agli amministratori, ma soltanto attraverso una console priva di controllo di funzione: il server la esegue [19]. Il controllo, in un certo senso, esisteva, ma stava nella console, nel fatto che solo un amministratore vedesse quel

pulsante. Tre osservazioni spostano infine il difetto fuori dal singolo gestore: il percorso non dice se un endpoint sia amministrativo, perché le funzioni riservate stanno spesso sotto `api/users` anziché sotto `api/admins` [19]; i verbi HTTP lasciati abilitati per inerzia riaprono l'operazione da una via non presidiata [21]; le versioni obsolete continuano a servire il gestore precedente al controllo [22].

OWASP colloca l'impatto tecnico nella fascia grave, con esiti che vanno dalla divulgazione alla perdita o corruzione dei dati, fino all'interruzione del servizio [19]. Il danno non scala però con il numero di identificatori, bensì con la potenza della funzione raggiunta: una BOLA restituisce un record per richiesta, mentre una BFLA su un endpoint di esportazione ottiene l'intera base dati con una chiamata sola; e poiché le funzioni riservate sono in prevalenza funzioni di scrittura, il difetto che espone l'esportazione degli utenti espone, sull'endpoint accanto, la loro cancellazione.

La prescrizione centrale riguarda il valore predefinito: negare ogni accesso e richiedere concessioni esplicite ai ruoli per ogni funzione [19], con un vantaggio che sta in come si manifesta l'errore, perché se il default è il diniego una dimenticanza produce un `403` che il primo test funzionale rivela, mentre se il default è il permesso la stessa dimenticanza non produce alcun segnale. Segue la revisione degli endpoint alla luce della logica di business e della gerarchia dei gruppi, lavoro che non ammette scorciatoie rispetto alla conoscenza del dominio. La terza è l'unica con una forma sintattica: i controller amministrativi devono ereditare da un controller astratto che applichi il controllo in base a ruolo o gruppo, misura che non aggiunge verifiche ma le riposiziona, perché ometterle richiede l'atto deliberato di collocare un gestore fuori dalla gerarchia, discontinuità visibile in revisione. Nessuna delle tre risolve però la copertura, perché il controllo protegge solo la superficie servita dal codice che lo contiene [22, 21].

2.2.6 API6:2023 – Unrestricted Access to Sensitive Business Flows

Esistono attacchi che non violano alcuna regola. Qui ogni singola chiamata è legittima: l'utente ha diritto di compierla, i controlli di autorizzazione funzionano, il codice si comporta come progettato, e il danno nasce dal numero. OWASP definisce vulnerabile l'endpoint che espone un flusso di business sensibile senza limitarne ade-

guatamente l'accesso, portando come esempi l'acquisto di un prodotto, la creazione di un commento e la prenotazione di una fascia oraria [23]. Segue l'affermazione che definisce davvero la categoria: il rischio di un accesso eccessivo cambia da settore a settore, e la creazione di post tramite script può costituire spam per un social network ed essere incoraggiata da un altro [23]. Nessuna proprietà del codice, del contratto o della configurazione rende dunque un endpoint vulnerabile ad API6: lo stesso identico endpoint è sicuro in un contesto e vulnerabile in un altro, e a decidere è il modello di business che gli sta dietro.

Coerentemente, la causa che OWASP indica non è tecnica: manca una visione d'insieme dell'API che ne supporti i requisiti di business [23]. Il difetto nasce nel raccordo fra chi progetta l'endpoint e chi conosce il valore economico di ciò che l'endpoint muove, e dove quel raccordo manca nessuno è nella posizione di accorgersene. Lo sfruttamento richiede di comprendere il modello di business, individuare i flussi sensibili e automatizzarne l'accesso, e soltanto l'ultimo passo produce traffico osservabile: il primo scenario lo illustra con una console da gioco annunciata in quantità limitata, acquistata da un programma eseguito il giorno del lancio e distribuito su indirizzi IP e località diverse [23], dove la distribuzione geografica non è un dettaglio di colore ma la contromisura dell'attaccante contro il limite per indirizzo, e spiega perché il rate limiting nella sua forma più semplice non basti.

Sull'impatto il documento OWASP è internamente incoerente: la tabella dei punteggi classifica l'impatto tecnico come moderato, mentre la descrizione degli impatti afferma che un impatto tecnico in generale non è atteso e porta come esempi di danno l'impossibilità per gli utenti legittimi di acquistare un prodotto e l'inflazione nell'economia interna di un videogioco [23]. È la seconda formulazione a corrispondere agli scenari descritti, che non producono né violazione di riservatezza né alterazione di dati né indisponibilità del servizio: ciò che cambia sta fuori dal sistema e si misura in scorte sottratte, ricavi persi o costi di acquisizione pagati per utenti inesistenti. Ne segue un effetto collaterale rilevante, perché un incidente che non produce indisponibilità né perdita di dati non attiva le procedure di risposta e viene percepito come problema commerciale anziché di sicurezza.

La mitigazione proposta ha una struttura a due livelli in cui l'ordine conta: prima si identificano, sul piano del business, i flussi che un uso eccessivo può danneggiare, e

solo dopo si sceglie il meccanismo tecnico adeguato al rischio individuato [23]. Nessuna categoria precedente prescrive una fase preliminare simile, perché in nessuna altra la configurazione corretta dipende dall'azienda che la adotta. I meccanismi elencati, dal fingerprinting del dispositivo al captcha, dall'individuazione di comportamenti non umani come il passaggio dal carrello all'acquisto in meno di un secondo fino al blocco dei nodi di uscita Tor, sono presentati come modi per rallentare le minacce automatizzate, non per impedirle, e la logica che li accomuna è economica: nessuno produce una decisione binaria affidabile, tutti spostano il costo dell'attacco fino a superare il profitto atteso.

2.2.7 API7:2023 – Server Side Request Forgery

Il traffico non viaggia in una sola direzione: verso servizi interni, archivi remoti e risorse indicate da chi interroga il servizio partono richieste che l'applicazione stessa emette, ed è su queste che agisce l'attaccante in API7:2023.

La causa immediata è l'assenza o l'inadeguatezza della validazione dell'URI fornito dal client, e sono i modelli di sviluppo moderni a incoraggiare l'accesso a URI di provenienza esterna: il difetto nasce quindi da una funzionalità voluta, perché nessuno introduce per sbaglio un endpoint che scarica un'immagine da un indirizzo scelto dall'utente. I due scenari del documento mostrano le due forme dell'attacco. Nella forma cieca un social network consente di indicare l'immagine del profilo tramite URL, e l'attaccante vi scrive un indirizzo interno:

```
POST /api/profile/upload_picture HTTP/1.1
Host: api.example.com
Content-Type: application/json

{"picture_url": "http://localhost:8080"}
```

Misurando il tempo di risposta deduce se la porta sia aperta, e scandisce così la rete interna [24]: l'informazione non arriva dal corpo della risposta, ma dalla sua latenza. Nella forma con ritorno un prodotto di sicurezza invia una richiesta di prova al webhook che l'utente sta configurando e gliene mostra la risposta, e l'attaccante vi indica l'indirizzo link-local del servizio di metadati dell'ambiente cloud —

`http://169.254.169.254/latest/meta-data/` nella convenzione più diffusa — ottenendone le credenziali [24]: il webhook rende comune il difetto, l'indirizzo a percorso noto lo rende grave. Gli esiti vanno dall'enumerazione dei servizi interni per scansione di porte alla divulgazione di informazioni, dall'aggiramento di firewall all'uso del server come proxy [24]. L'elemento che distingue l'SSRF è il cambio di perimetro: l'attaccante non ottiene un dato, ottiene una posizione, quella del server, e il valore dell'attacco dipende perciò da cosa sia raggiungibile dall'interno. I servizi di metadati cloud convertono poi l'SSRF da ricognizione a compromissione, perché rispondono a un indirizzo noto senza autenticazione ed espongono le credenziali del ruolo associato alla macchina [25].

Sul piano della rete OWASP raccomanda di isolare il meccanismo di recupero delle risorse, che di norma mira a risorse remote e non interne: è la difesa più robusta perché non dipende dalla correttezza della validazione, dato che se il componente che effettua le richieste in uscita non ha rotta verso la rete interna un URL malevolo non produce nulla. Sul piano applicativo sono prescritte liste di elementi ammessi su tre dimensioni — origini remote attese, schemi di URL e porte, tipi di media accettati — ciascuna delle quali blocca una famiglia di aggiramenti diversa, cui seguono la disabilitazione delle redirezioni, un parser di URL collaudato e il divieto di restituire al client le risposte grezze [24]: quest'ultima non impedisce l'attacco, ma ne degrada l'esito dalla forma con ritorno a quella cieca. Va infine registrata un'ammissione che nessun'altra categoria contiene: il rischio SSRF non può sempre essere eliminato del tutto [24], perché funzionalità e vulnerabilità coincidono.

2.2.8 API8:2023 – Security Misconfiguration

Un controllo che manca, un limite non impostato, un URL non validato: ciascuna delle categorie precedenti indica un punto preciso in cui il difetto risiede. La Security Misconfiguration non ha un punto: OWASP dichiara che la configurazione insicura può verificarsi a qualunque livello dello stack, dalla rete all'applicazione, e i suoi otto indicatori attraversano tutti quei livelli, dall'hardening assente ai verbi HTTP lasciati abilitati, dal TLS mancante ai messaggi di errore che espongono tracce dello stack [21]. Hanno una sola cosa in comune: nessuno è un errore di logica applicativa,

perché il codice fa quello che deve ed è l'ambiente in cui gira a produrre l'esposizione. Lo conferma la particolarità tassonomica: fra le sette voci CWE associate alla categoria compare CWE-16 *Configuration*, di cui MITRE dichiara proibita la mappatura, invitando ad analizzare piuttosto il comportamento che la configurazione ha determinato [26]. Se ne ricava il criterio: la configurazione non è una debolezza, è il meccanismo attraverso cui una debolezza viene introdotta, sicché una politica CORS permissiva non è insicura in astratto, lo diventa perché produce un difetto di controllo di accesso.

La causa strutturale è che la configurazione non risiede in un luogo unico né sotto una responsabilità unica: si distribuisce fra codice, file di orchestrazione, gateway, impostazioni cloud e valori predefiniti delle librerie, ciascuno con un ciclo di vita proprio, e un valore corretto al rilascio può cessare di esserlo dopo un aggiornamento che nessuno ha collegato alla sicurezza. Il primo scenario mostra il predefinito pericoloso, ed è riconoscibilmente il caso Log4Shell: un'utilità di logging supporta le ricerche JNDI abilitate per impostazione predefinita, e quando espande un'intestazione contenente un riferimento JNDI a un server LDAP controllato dall'attaccante ne scarica ed esegue l'oggetto malevolo [21], senza che nella catena vi sia alcun difetto del codice applicativo. Il secondo mostra il meccanismo opposto, l'impostazione assente: un social network espone i messaggi privati tramite `/dm/user_updates.json` e, non inviando l'intestazione `Cache-Control`, lascia che le conversazioni finiscano nella cache del browser, e qui non c'è nulla da disabilitare, c'è una direttiva che nessuno ha inviato. Resta poi la causa più diffusa, che l'elenco mette al secondo posto: patch mancanti e sistemi non aggiornati [21], difetto di manutenzione più che di configurazione.

Le configurazioni insicure, scrive OWASP, non solo espongono dati sensibili degli utenti, ma anche dettagli del sistema che possono condurre alla compromissione completa del server [21]: è la seconda parte a giustificare l'impatto severo, e i due scenari ne mostrano gli estremi, conversazioni private da un lato ed esecuzione di codice arbitrario dall'altro. Ciò che distingue la Security Misconfiguration è però la natura abilitante del difetto, che raramente costituisce l'attacco e quasi sempre la condizione che ne rende praticabile un altro: una traccia di stack non compromette nulla di per sé, ma rivela versioni e percorsi interni e abbassa il costo della ricognizione,

mentre una politica di uscita permissiva trasforma un SSRF cieco in uno con ritorno. Vi si aggiunge l'ampiezza, perché un difetto di configurazione, essendo trasversale, non riguarda un endpoint ma tutti quelli serviti da quel componente.

Sul piano del processo OWASP chiede un hardening ripetibile, una revisione delle configurazioni sull'intero stack e un processo automatizzato che ne valuti l'efficacia in tutti gli ambienti, la cui implicazione comune conviene rendere esplicita: la configurazione va trattata come artefatto versionato e verificabile, non come stato accumulato di una macchina. Sul piano dei controlli le misure riguardano il canale cifrato verso ogni componente, i verbi HTTP da restringere, la politica CORS e le intestazioni di sicurezza, i tipi di contenuto accettati e gli schemi dei payload di risposta, comprese quelle di errore [21]; due meritano una nota, perché il canale cifrato è prescritto anche per le API interne, il che chiude l'assunto per cui il traffico interno sarebbe fidato, e lo schema dichiarato per le risposte di errore non è documentazione, ma un vincolo che a runtime impedisce la fuga di informazioni diagnostiche.

2.2.9 API9:2023 – Improper Inventory Management

Correggere un difetto presuppone sapere che esiste il componente che lo ospita, e fin qui quel componente è sempre stato noto: un handler, un limite, una configurazione. API9:2023 riguarda invece ciò che l'organizzazione non sa di avere: il difetto non è nel codice ma nella visibilità, e OWASP lo definisce come scarsa catalogazione degli host e delle versioni di un'API, articolandolo in due punti ciechi [22]. Il primo è quello della documentazione, perché restano senza risposta esplicita le domande su quale ambiente ospiti un'API, chi debba avervi accesso di rete e quale versione sia in esecuzione, e non esistono né un piano di dismissione né un inventario aggiornato degli host. Il secondo è quello dei flussi di dati, e ricorre quando dati sensibili raggiungono terze parti senza motivazione di business, approvazione o visibilità su cosa venga condiviso [22]. Anche qui il riferimento tassonomico è una voce non mappabile, CWE-1059 [27]: come per la Security Misconfiguration, la categoria non nomina un comportamento del prodotto, e nel suo caso non è un'imprecisione della tassonomia ma una proprietà del difetto, perché l'assenza di un inventario non è qualcosa che il software faccia.

La causa è un'asimmetria fra due costi: esporre una nuova istanza richiede pochi minuti, catalogarla e prevederne la dismissione richiede un processo che nessuna pipeline esegue da sola, e ogni ambiente di collaudo è però un'API completa e raggiungibile, mentre l'inventario che dovrebbe elencarlo è un documento aggiornato a mano. Il primo scenario ne mostra la conseguenza: un social network protegge il recupero della password dalla forza bruta, ma il meccanismo non è nel codice dell'API, bensì in un componente interposto fra il client e l'host ufficiale, e un ricercatore individua `beta.api.socialnetwork.owasp.org`, che esegue la stessa API senza esserne coperto, e reimposta la password di qualunque utente indovinando un token di sei cifre [22]: il controllo esisteva ed era corretto, ma non copriva tutti gli host che servivano quel codice. Il secondo riguarda l'altro punto cieco, con un'applicazione malevola che ottiene il consenso di 270.000 utenti ma, attraverso di essi, accede alle informazioni private di 50.000.000 di contatti: qui non c'è alcun host dimenticato, manca la conoscenza di dove i dati finiscano.

OWASP classifica l'impatto tecnico come moderato, ma il dettaglio decisivo è che versioni diverse della stessa API sono talvolta collegate alla medesima base di dati [22]: l'host dimenticato non è una copia isolata, è una seconda porta sugli stessi dati. Ne discende il tratto distintivo, perché API9 non introduce una vulnerabilità propria ma rende raggiungibili quelle altrui e ne impedisce la correzione, dato che una patch si applica a ciò che si sa di avere in esecuzione; il giudizio di impatto moderato è quindi fuorviante, perché la gravità effettiva è quella del difetto peggiore che l'host non inventariato continua a servire.

Le contromisure muovono tutte dallo stesso presupposto, che la visibilità vada prodotta e non presupposta. Le prime sono di inventario — gli host, con ambiente, accesso di rete e versione, e i servizi integrati, con ruolo e sensibilità dei dati — cui segue la documentazione dell'API, che deve coprire autenticazione, errori, redirezioni, limitazione di frequenza, politica CORS ed endpoint. La misura più incisiva è però la successiva: generare la documentazione in modo automatico, con standard aperti e dentro la pipeline di integrazione continua [22], differenza non di grado ma di natura, perché trasforma la documentazione da documento parallelo, che diverge appena qualcuno dimentica di aggiornarlo, in artefatto prodotto dalla compilazione. Restano le prescrizioni sulle versioni: applicare le misure di sicurezza esterne a tutte

quelle esposte e non alla sola produzione, che è ciò che il primo scenario mostra disatteso, e non usare dati di produzione fuori dalla produzione.

2.2.10 API10:2023 – Unsafe Consumption of APIs

Il perimetro di un servizio non coincide con il codice che lo compone: ogni dipendenza esterna è una superficie che non gli appartiene. API10:2023 rovescia perciò la prospettiva dell'attacco diretto, presupposta dalle nove categorie precedenti: l'attaccante non colpisce l'API bersaglio ma un servizio da cui essa dipende, e il danno arriva per un canale che l'applicazione considera fidato; le note di rilascio dichiarano di aver introdotto la categoria perché gli attaccanti hanno cominciato a compromettere i servizi integrati anziché le API del bersaglio [28]. La causa che OWASP individua è psicologica prima che tecnica: gli sviluppatori si fidano dei dati ricevuti da terze parti più di quanto si fidino dell'input dell'utente, e adottano standard più deboli quanto a trasporto, autenticazione e validazione [29]. Le cinque condizioni di vulnerabilità sono canale non cifrato, dati non validati, redirezioni seguite ciecamente, risorse non limitate e timeout assenti.

La causa strutturale è un confine di fiducia collocato nel punto sbagliato. Un servizio di terze parti occupa una posizione ambigua: è esterno all'applicazione, ma scelto deliberatamente, raggiungibile su un canale che essa ha configurato e spesso offerto da un'azienda reputata affidabile, e la sua risposta attraversa così un confine che nessuno ha presidiato, perché nessuno lo ha riconosciuto come tale. Il primo scenario mostra l'iniezione differita: un'API si affida a un servizio esterno per arricchire gli indirizzi commerciali degli utenti e ne salva i dati in una base di dati SQL, e gli attaccanti vi memorizzano un payload di SQL injection associato a un'attività creata da loro, poi inducono l'API a prelevarla [29]; ciò che lo distingue da un'iniezione ordinaria è che l'attaccante non invia mai il payload all'API bersaglio, e sanificare l'input degli utenti non serve, perché quell'input è un innocuo nome di attività. Il secondo mostra la redirezione seguita ciecamente: un'API invia informazioni mediche a un fornitore su connessione cifrata, gli attaccanti lo compromettono ed esso risponde con un `308 Permanent Redirect`, sicché l'API ripete la richiesta identica, dati sensibili compresi, verso il loro server, e la fuga avviene

per un comportamento predefinito del client HTTP che nessuno ha valutato come decisione di sicurezza. Il terzo scenario è il più istruttivo, perché la terza parte non deve nemmeno essere compromessa: un attaccante crea un repository Git denominato `';dropdb;--`, e l'applicazione che vi si integra ne usa il nome per costruire una query SQL ritenendolo sicuro [29], avendo confuso l'autenticità del canale con l'affidabilità del contenuto.

L'impatto varia a seconda di ciò che l'API bersaglio fa con i dati prelevati — esposizione di informazioni sensibili, molti tipi di iniezione o denial of service [29] — ed è classificato come severo. Ciò che caratterizza la categoria è che il difetto non ne determina l'entità, la quale è funzione di dove il dato non validato finisce: un'informazione prelevata da un fornitore e memorizzata può essere riletta mesi dopo da un componente che la tratta come dato interno, perché proviene dalla base di dati dell'organizzazione. Le contromisure prescritte sono quattro: valutare la postura di sicurezza dei fornitori al momento della scelta, imporre un canale di trasporto sicuro, validare e sanificare i dati ricevuti dalle API integrate, mantenere una lista delle posizioni verso cui esse possono redirigere [29]. La terza è il nucleo della categoria ed è formulata negli stessi termini riservati all'input dell'utente: la contromisura consiste nell'abolire la distinzione fra le due sorgenti. Due delle cinque condizioni — timeout e limiti sulle risposte — restano però prive di contromisura corrispondente; quando il contratto del fornitore è noto, la risposta può essere validata rispetto a uno schema, che non intercetta un payload conforme ma trasforma la fiducia implicita in un vincolo dichiarato.

2.2.11 Sintesi: dove si trova il difetto

Le categorie analizzate in questo capitolo si differenziano anche per il punto in cui è possibile individuare il difetto. In alcuni casi la vulnerabilità può essere individuata direttamente nel codice, nelle configurazioni o nelle chiamate effettuate dal sistema. In altri casi, invece, queste informazioni mostrano solo una parte del problema e non permettono di stabilire se i controlli presenti funzionino correttamente.

Esistono infine categorie in cui le informazioni necessarie non sono presenti direttamente nel codice o nella configurazione, ma dipendono dal comportamento

del sistema o dal contesto in cui viene utilizzato.

Questa distinzione è alla base del Capitolo 3, dove le categorie vengono classificate in base a ciò che può essere rilevato direttamente e alle informazioni aggiuntive necessarie per individuarle.

CAPITOLO 3

Metodo di ricerca

Questo capitolo descrive lo strumento realizzato per rilevare le vulnerabilità dell'OWASP API Security Top 10 (2023) e delle configurazioni infrastrutturali. Dopo aver introdotto il criterio con cui le categorie vengono classificate e i requisiti che ne derivano per lo strumento, il capitolo ne presenta l'architettura e descrive i moduli di rilevazione: prima quello per le configurazioni infrastrutturali, poi quelli per le categorie ibride e per le categorie tecniche. La valutazione dello strumento è oggetto del Capitolo 4.

3.1 Approccio metodologico

La rassegna del capitolo precedente restituisce un quadro disomogeneo: le categorie dell'OWASP API Security Top 10 differiscono profondamente per il luogo e il modo in cui la violazione si manifesta. Alcune lasciano una traccia strutturale negli artefatti di progetto (codice applicativo, definizioni infrastrutturali, specifica dell'interfaccia), altre vi lasciano solo una condizione parziale o presuppongono di sapere quale uso dell'applicazione sia legittimo. È la distinzione fra *vulnerabilità tecniche* e *vulnerabilità di logica di business* [30, 31], alla quale la stessa OWASP riconduce la resistenza di queste ultime al rilevamento automatico [23].

3.1.1 Il criterio di classificazione

Il criterio rende operativa la distinzione appena richiamata. Per ogni categoria si individua la proprietà di sicurezza la cui violazione la definisce, attenendosi alla formulazione ufficiale OWASP [4], alle debolezze CWE associate e alla caratterizzazione emersa dalla rassegna.

Una categoria è detta *tecnica* quando esiste un predicato sugli artefatti di progetto che sia al tempo stesso necessario e sufficiente a stabilire la violazione, una volta fissati i parametri dichiarati dell'analisi. È detta *ibrida* quando esiste un predicato necessario ma non sufficiente: l'analisi statica riconosce una condizione senza la quale la violazione non può sussistere, ma la cui presenza non basta a stabilirla. È detta infine *di logica di business* quando non esiste alcun predicato necessario esprimibile sugli artefatti, poiché la proprietà violata è definita per riferimento all'intento applicativo.

È necessario delimitare quali parametri sia lecito fornire all'analisi: si ammettono unicamente configurazioni generali valide per l'intero sistema (come l'uso di canali cifrati o limiti di traffico attesi). Si escludono invece regole puntuali sui permessi degli utenti rispetto alle singole risorse, per evitare di inserire a priori la logica di autorizzazione che l'analisi stessa dovrebbe verificare.

Il residuo delle categorie ibride non è omogeneo, e distinguerne la natura serve a capire che cosa la parte statica non può vedere. Un residuo è *dinamico* quando la proprietà mancante si manifesta nella risposta del sistema a una sollecitazione mirata, tipicamente l'invio della stessa richiesta con identità distinte. È *di perimetro* quando è colmabile estendendo l'analisi a un livello non ancora esaminato: il limite riguarda l'applicazione dell'analisi statica a un solo livello, non l'approccio in sé. È *di dichiarazione* quando richiede che qualcuno espliciti la configurazione attesa o il contenuto di un accordo esterno. È *irriducibile* quando nessuna osservazione del sistema, statica o dinamica, può restituirlo.

Il criterio presuppone infine che il sistema da valutare sia noto. La condizione non vale per API9:2023, che descrive l'esistenza di sistemi che l'organizzazione ignora di possedere: la categoria resta perciò fuori dalla tripartizione.

3.1.2 Requisiti derivati per lo strumento di rilevazione

La classificazione non descrive uno strumento, ma vincola ciò che uno strumento adeguato deve saper fare. I requisiti che seguono sono formulati come capacità richieste, indipendentemente dalle tecnologie; la loro traduzione in un'architettura concreta è mostrata nella Sezione 3.2 e nelle sezioni dedicate ai singoli moduli.

Il primo requisito è l'analisi su più livelli. Le proprietà individuate dal criterio non risiedono tutte nello stesso luogo: alcune si manifestano nel codice applicativo, altre nelle definizioni infrastrutturali, altre nella specifica dell'interfaccia, e nessun livello è da solo sufficiente. Uno strumento che esamini un solo livello non può quindi coprire il perimetro, e ciascun livello richiede una rappresentazione e una tecnica di ispezione proprie.

Il secondo è la correlazione fra i livelli. Alcune condizioni emergono solo confrontando due livelli. Un percorso presente nel codice ma assente dal contratto, per esempio, non è un errore in nessuno dei due, ma indica un endpoint esposto e non documentato. Allo stesso modo, un limite assente nel codice è un problema solo se non lo applica nemmeno l'infrastruttura. Lo strumento deve quindi combinare i segnali dei diversi livelli.

Il terzo è la verifica dinamica: per le categorie che richiedono un riscontro a runtime, lo strumento deve poter inviare richieste reali all'API e analizzarne le risposte. L'analisi del codice rileva infatti solo un rischio potenziale, che rimarrebbe incerto senza una conferma attiva.

Il quarto è la configurabilità dei parametri: i valori e le soglie utilizzati dall'analisi devono essere espliciti e modificabili dall'utente, per evitare che impostazioni predefinite e cablate nelle regole impongano scelte arbitrarie come se fossero vincoli universali.

Il quinto è la tracciabilità: ogni segnalazione deve essere riconducibile alla categoria di cui costituisce evidenza e al livello di analisi che l'ha generata, perché una segnalazione priva di tale riconduzione può indicare un difetto reale ma non permette di valutare la copertura della categoria.

3.2 Architettura del sistema

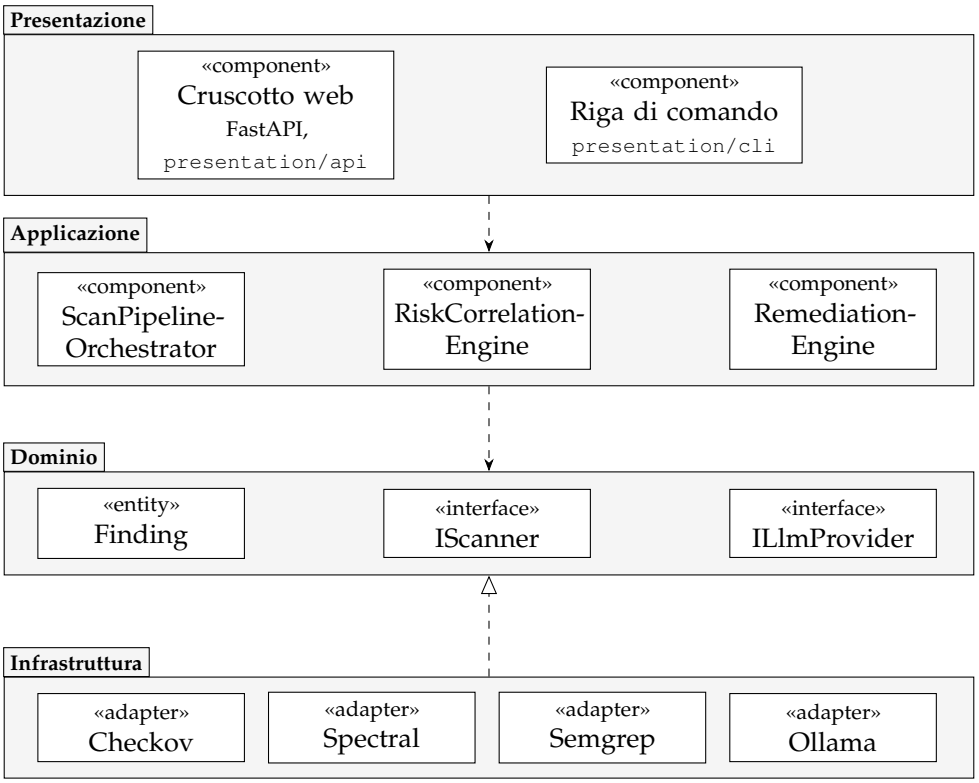


Figura 3.1: Diagramma UML dei componenti del sistema, raggruppati per livello architetturale. Le frecce tratteggiate indicano dipendenze; quella con la punta triangolare vuota indica la realizzazione delle interfacce di dominio da parte degli adapter.

Il sistema è organizzato secondo i principi della *Clean Architecture*: un livello di dominio che definisce entità e contratti, un livello applicativo che orchestra l'analisi, un livello infrastrutturale che incapsula gli strumenti esterni e un livello di presentazione, a riga di comando o tramite un cruscotto web, che espone i risultati (Figura 3.1).

In risposta alla tracciabilità richiesta nella Sottosezione 3.1.2, ogni anomalia, qualunque strumento l'abbia prodotta, viene ricondotta a un'unica entità di dominio, il *Finding*, che ne registra la sorgente e la regola, la posizione con una chiave di correlazione, la categoria, la severità (da `CRITICAL` a `INFO`) e un grado di confidenza fra 0 e 1. Vi si aggiunge la *natura* del rischio: un'*esposizione* quando la configurazione apre un varco sfruttabile direttamente, un *irrobustimento* quando manca una difesa in profondità che da sola non costituisce un accesso; se la regola non permette di stabilirlo, la segnalazione resta non classificata.

Gli strumenti esterni sono avvolti da *adapter* che realizzano l'interfaccia di dominio `IScanner`, eseguono lo strumento e ne traducono l'uscita in Finding: Checkov [32] per le definizioni infrastrutturali, Semgrep per il codice applicativo e Spectral per la specifica dell'interfaccia. L'orchestratore conosce gli scanner solo attraverso questa interfaccia: li esegue sul repository bersaglio e consegna le segnalazioni al motore di correlazione, che le aggrega per chiave e assegna a ciascuna un punteggio di rischio. Aggiungere un controllo significa registrare un nuovo adapter, senza modificare l'orchestratore.

3.3 Rilevamento delle IaC misconfiguration

Questa sezione applica il metodo alla prima delle classi in esame, la *IaC misconfiguration* (Sottosezione 2.1.1): le configurazioni difettose dichiarate in file ispezionabili prima del rilascio. Qui tali file sono la definizione dell'infrastruttura come codice (i manifest Terraform, affiancati da Dockerfile e manifest Kubernetes gestiti dal medesimo strumento) e l'analisi è interamente statica: nessuna risorsa viene creata né interrogata.

Dei quattro scenari con cui OWASP illustra la categoria, due ricadono nel perimetro del sistema e due ne restano fuori. Rientrano il secondo e il quarto, perché in entrambi la configurazione difettosa è dichiarata in un file ispezionabile — la direttiva che abilita il directory listing nella configurazione del server, i permessi di condivisione fissati nel manifest della risorsa di storage — e può quindi essere individuata prima del rilascio. Restano fuori il primo e il terzo: le applicazioni di esempio dimenticate in produzione non compaiono in alcun file del progetto e presuppongono un inventario degli host effettivamente in esecuzione, mentre i messaggi di errore troppo dettagliati emergono soltanto dalle risposte prodotte a runtime.

3.3.1 Lo scanner Checkov e la traduzione nel modello di dominio

Checkov [32] è un analizzatore statico per Infrastructure-as-Code: legge i file di definizione dell'infrastruttura, fra cui i manifest Terraform [33], e vi applica un catalogo di controlli predefiniti. Ogni controllo verifica una proprietà di sicurezza su

una risorsa — per esempio che un bucket di storage non conceda la lettura pubblica — e restituisce un esito superato o fallito, con il file e le righe della risorsa interessata. Ogni controllo è però identificato solo da un codice opaco, come `CKV_AWS_20`, che da solo non dice né a quale tema si riferisca né quanto sia grave.

Lo scenario in perimetro riguarda i permessi di condivisione aperti su risorse di storage. Un bucket S3 può diventare pubblico per tre vie — la lista di controllo d'accesso, la policy del bucket e il mancato blocco dell'accesso pubblico — e Checkov ha un controllo per ciascuna. Sulle stesse risorse, però, ne valuta anche altri che non aprono alcun varco, come la registrazione degli accessi, il versioning o la cifratura, collocandoli sullo stesso piano dei primi. Distinguere ciò che espone un varco da ciò che aggiunge una difesa in profondità è essenziale per questo lavoro, ma in Checkov questa distinzione non esiste: la introduce il sistema, con un proprio catalogo delle policy descritto nel seguito. La Tabella 3.1 mette i due gruppi a confronto sulla stessa risorsa di storage.

Policy	**Risorsa Terraform**	**Condizione verificata**
Esposizione: la configurazione apre un varco		
CKV_AWS_20	`aws_s3_bucket`	ACL che concede lettura pubblica
CKV_AWS_53–56	`aws_s3_bucket_public_access_block`	i quattro flag del blocco dell'accesso pubblico non attivi
CKV_AWS_70	`aws_s3_bucket_policy`	policy che concede un'azione a qualunque principal
CKV2_AWS_6	`aws_s3_bucket`	bucket privo di un blocco dell'accesso pubblico attivo
CKV2_AWS_65	`aws_s3_bucket_ownership_controls`	liste di controllo d'accesso non disabilitate
Irrobustimento: manca una difesa, senza aprire un varco		
CKV_AWS_18	`aws_s3_bucket`	registrazione degli accessi assente
CKV_AWS_21	`aws_s3_bucket`	versioning non abilitato
CKV_AWS_145	`aws_s3_bucket`	cifratura non basata su chiave KMS
CKV_AWS_144	`aws_s3_bucket`	replica fra regioni assente
CKV2_AWS_61, 62	`aws_s3_bucket`	ciclo di vita e notifiche di evento assenti

Tabella 3.1: Controlli di Checkov 3.2.459 sulla stessa risorsa di storage, divisi fra quelli che segnalano un'*esposizione* e quelli che segnalano un mancato *irrobustimento*: una distinzione che lo strumento non fa e che il sistema introduce con il proprio catalogo. Le denominazioni sono tradotte dai nomi ufficiali dei controlli.

L'adapter esegue Checkov sulla sola directory bersaglio richiedendo l'uscita in formato JSON, e traduce ciascun controllo fallito in un Finding. La categoria

è derivata tramite parole chiave da identificativo, nome ufficiale e tipo di risorsa. Natura, severità ed esposizione pubblica provengono invece dal *catalogo delle policy*: un file YAML esterno che costituisce parametro dell'analisi e non costante del codice, in accordo con la configurabilità richiesta nella Sottosezione 3.1.2. Il catalogo risolve ciascun controllo fallito provando quattro regole in ordine: la mappatura esplicita dell'identificativo, il prefisso della famiglia di controlli, le parole chiave del nome ufficiale e, se nessuna si applica, un ripiego che lascia la segnalazione non classificata con severità `MEDIUM`.

La confidenza del Finding non misura il controllo, che è deterministico sul file, ma quanto è stata precisa la sua interpretazione nel catalogo. L'adapter registra inoltre il contesto del rischio, cioè l'esposizione a Internet o il coinvolgimento di dati sensibili, e usa come chiave di correlazione il nome della risorsa, così da raggruppare i rilievi riferiti alla stessa entità.

3.3.2 Correlazione e punteggio di rischio

Il motore di correlazione indicizza le segnalazioni per chiave (nel caso delle segnalazioni infrastrutturali, il nome della risorsa) e fonde quelle che la condividono in un'unica voce. La rappresentante è scelta per natura e poi per severità: un'esposizione precede una segnalazione non classificata, che precede un irrobustimento; a parità di natura prevale la severità maggiore. Un controllo di natura inferiore non altera mai la severità della rappresentante e resta come sotto-elemento della voce, con la propria natura e severità: nulla viene scartato, e una sola esecuzione restituisce l'intero insieme delle condizioni da correggere.

Il punteggio di rischio è un modello pesato proprio, perché il CVSS richiede un vettore che nessuno degli strumenti integrati produce:

$$R = \min\big(10,\ 0{,}6\,S + 0{,}2 \cdot 10\,C + 0{,}2\,X\big), \tag{3.3.1}$$

Il punteggio combina tre termini: la severità S della segnalazione, la confidenza $C \in [0, 1]$ dichiarata dalla sorgente — che sale a 1 quando un riscontro a runtime conferma la segnalazione — e il contesto X, che pesa l'esposizione a Internet e il coinvolgimento di dati sensibili. Per gli irrobustimenti X è nullo, perché una linea di difesa mancante non è di per sé un vettore d'accesso. I valori di severità, i contributi

di contesto e le mappe di confidenza delle diverse sorgenti sono predefiniti nel codice ma risiedono in un file YAML di configurazione, in accordo con la configurabilità richiesta nella Sottosezione 3.1.2.

Il cruscotto ordina le voci per natura e poi per punteggio decrescente, mostrato accanto alla severità, e sotto ciascuna elenca i controlli aggregati. Ogni voce riceve un'indicazione correttiva, tratta da una base di conoscenza locale o, in sua assenza, generata da un modello linguistico eseguito localmente tramite Ollama.

3.4 Categorie ibride: analisi statica e residuo dinamico

Le categorie raccolte in questa sezione sono quattro: la Broken Object Level Authorization (API1), la Broken Authentication (API2), la Broken Object Property Level Authorization (API3) e la Broken Function Level Authorization (API5). Tre riguardano l'autorizzazione a granularità diverse, cioè l'oggetto a cui si accede, le sue singole proprietà e la funzione che si invoca; la quarta riguarda il passo che le precede, il riconoscimento di chi effettua la richiesta. Secondo il criterio della Sottosezione 3.1.1 sono tutte categorie ibride, e per ciascuna il sistema separa la parte riconoscibile staticamente dal residuo.

Per ciascuna esiste infatti un predicato sugli artefatti necessario ma non sufficiente. Un oggetto recuperato senza legarlo al chiamante, un token decodificato senza verificarne la firma, un oggetto restituito con tutti i suoi campi riservati, una funzione amministrativa priva di controllo di ruolo sono condizioni senza le quali la violazione non può esistere, e l'analisi statica le trova. Quando il controllo è presente, invece, il codice non dice se funzioni: il filtro sul proprietario può usare il campo sbagliato, un token scaduto può essere accettato, un controllo di ruolo può non bloccare nulla. Proprietà di questo tipo si osservano solo nella risposta del sistema a una sollecitazione mirata, di solito la stessa richiesta inviata con identità diverse, e formano il residuo che il criterio chiama *dinamico*.

Tutti i moduli distinguono tre tipi di controllo, che nel codice si somigliano perché spesso sono decoratori o middleware applicati alla stessa rotta: l'autenticazione stabilisce chi è il chiamante, il controllo di proprietà se l'oggetto richiesto gli appartiene, il controllo di ruolo che cosa gli è permesso fare. Ogni modulo cerca quello perti-

nente alla propria categoria (il legame fra oggetto e chiamante per la BOLA, il ruolo attorno a una proprietà per la BOPLA, il ruolo sulla funzione per la BFLA), mentre il modulo della Broken Authentication esamina il meccanismo stesso che riconosce il chiamante. Per le categorie di autorizzazione la sola autenticazione non vale mai come protezione, perché qualunque utente registrato la supera. Ciò che è protetto, come impone il criterio (Sottosezione 3.1.1), va dedotto dal codice e dal contratto.

Struttura comune dei rilevatori I quattro moduli non condividono un'unica architettura, ma seguono due impianti, che si distinguono anzitutto per il trattamento del residuo dinamico; la BOLA li usa entrambi, perché affianca a un rilevatore statico un livello dinamico. Li si descrive qui una volta sola, lasciando alle sottosezioni ciò che è proprio di ciascuna categoria.

Il primo impianto, *statico a livelli*, è quello della BOLA e della BFLA. Analizza prima il codice sorgente e, quando è disponibile, il contratto OpenAPI; la BFLA vi aggiunge i file di configurazione. Il codice non viene cercato come testo, ma letto attraverso l'albero sintattico prodotto da tree-sitter [34], così che commenti, formattazione e decoratori non ne alterino l'interpretazione: ogni funzione che gestisce una rotta viene associata al percorso, ai metodi e ai controlli che la proteggono, e su questa mappa operano le regole. Il contratto serve soprattutto come termine di confronto: un'operazione che la specifica dichiara protetta e che il codice non protegge smentisce una garanzia promessa, mentre un endpoint presente nel codice ma assente dal contratto rivela una superficie non documentata. È la correlazione fra livelli prevista dai requisiti. Quando più regole colpiscono lo stesso endpoint, le segnalazioni vengono deduplicate tenendo quella che diagnostica il difetto in modo più preciso, secondo un ordine di priorità proprio di ciascun modulo e, a parità di priorità, preferendo quella ricavata dal codice, che punta a una riga precisa. Le regole che descrivono un altro aspetto del difetto non vengono deduplicate, per non nasconderne una parte. Per la BFLA questo impianto è l'intero modulo: riconosce l'assenza del controllo di ruolo ma non invoca le funzioni per accertarla, e il residuo dinamico rimane aperto.

Il secondo impianto, *a fasi*, è quello della Broken Authentication e della BOPLA, e arriva fino al sistema in esecuzione: una fase di scoperta raccoglie ciò che codice,

contratto e traffico dicono del bersaglio, una fase di sintesi ne ricava un modello, una batteria di prove dinamiche lo mette alla prova e un rapporto finale ne raccoglie gli esiti, riassunti in un punteggio di robustezza. I due moduli condividono parte dell'implementazione, e anche il livello dinamico della BOLA procede allo stesso modo, dalla scoperta degli endpoint fino all'attacco. Il peso della parte statica, però, cambia. Nella BOLA e nella BOPLA la segnalazione statica è un'ipotesi: le prove dinamiche inviano richieste reali con identità di ruolo diverso e il loro esito torna sulla segnalazione, confermandola o smentendola. Nella Broken Authentication il rapporto si rovescia, perché la parte statica serve soprattutto a ricostruire il modello di autenticazione che orienta le prove, e una sola regola statica copre il caso in cui il sistema non si lascia interrogare.

I moduli non producono però segnalazioni nello stesso formato. Solo il livello dinamico della BOLA genera il Finding di dominio della Sezione 3.2, che raggiunge il motore di correlazione (Sottosezione 3.3.2) attraverso la chiave di metodo e percorso normalizzato; gli altri moduli producono un rapporto in forma propria. Per questo nel cruscotto i moduli vengono eseguiti in sequenza e presentati modulo per modulo.

3.4.1 API1 – Il legame fra oggetto e chiamante

Una *Broken Object Level Authorization* [5] (Sottosezione 2.2.1) si verifica quando un endpoint riconosce chi lo invoca, ma non controlla che l'oggetto richiesto appartenga a quell'utente. Un esempio tipico è un endpoint come `GET /api/orders/{id}`: è protetto dal login, ma a un utente basta sostituire nel percorso l'identificativo del proprio ordine con quello di un ordine altrui per leggerne il contenuto.

Il livello statico cerca nel codice gli handler che accedono a un oggetto senza legarlo all'identità del chiamante; il livello dinamico copre i legami presenti ma sbagliati, come un campo che non esprime davvero l'appartenenza o un proprietario che il client può falsificare. I due livelli comunicano attraverso una chiave comune, formata dal metodo e dal percorso normalizzato dell'endpoint.

Il livello statico Il livello statico analizza applicazioni Flask, FastAPI ed Express. Per prima cosa stabilisce quali parametri di percorso fanno riferimento a un oggetto:

sono quelli tipizzati come interi e quelli il cui nome richiama un identificativo, come `id`, `order_id`, `uuid` o `slug`. È la stessa euristica usata a runtime dalla fase di discovery, e permette di ignorare i parametri che non indicano un oggetto, come il termine di una ricerca.

Poi legge istruzione per istruzione il corpo di ogni handler, ripulito da commenti e docstring, e ne ricava i *fatti di ownership*. Questi fatti rispondono a tre domande: dove un identificativo del percorso viene usato per leggere o modificare un oggetto, anche passando per una variabile intermedia; dove viene letta l'identità del chiamante, per esempio `current_user` in Flask o `req.user` in Express; quali legami collegano l'oggetto a quell'identità. Un legame può essere una query che filtra anche sull'identità, un confronto fra il proprietario dell'oggetto e l'utente corrente, la chiamata a una funzione che verifica la proprietà oppure un decoratore dedicato.

Per questi controlli sono state definite tre euristiche: la prima riguarda le condizioni composte, che vengono spezzate nelle singole clausole: così una condizione che nomina sia l'identità sia l'identificativo, senza però metterli in relazione, non viene scambiata per un controllo. La seconda riguarda i percorsi annidati, dove contano anche i legami fra identificativi: un task cercato all'interno di un progetto già validato è protetto, un task cercato con il solo identificativo no. La terza riguarda le funzioni di supporto definite nello stesso file. Spesso sono proprio loro a recuperare l'oggetto e a controllarne il proprietario, perciò il modulo ne riassume il comportamento e lo attribuisce all'handler che le chiama.

Le regole statiche Sui fatti di ownership operano le cinque regole della Tabella 3.2. OA-003 ha la confidenza più alta: se un metodo della stessa risorsa filtra sul proprietario e un altro no, è il codice stesso a mostrare che il controllo era un requisito noto.

L'ordine di priorità è OA-003 $>$ OA-004 $>$ OA-001 $>$ OA-002; OA-005, che descrive l'identificativo e non il controllo, resta fuori dalla selezione. Le segnalazioni sono raggruppate per file oltre che per chiave, perché due servizi dello stesso repository possono esporre lo stesso percorso con implementazioni diverse.

Regola	Condizione segnalata
OA-001	handler autenticato che accede all'oggetto con il solo identificativo inviato dal client
OA-002	lo stesso accesso in un handler privo di qualsiasi contesto di identità
OA-003	handler della stessa risorsa incoerenti: uno filtra sul proprietario, un altro no
OA-004	oggetto annidato cercato con il solo identificativo, senza passare per l'oggetto padre
OA-005	identificativo numerico, quindi prevedibile, esposto senza autenticazione

Tabella 3.2: Le cinque regole statiche del modulo della Broken Object Level Authorization, tutte sul codice sorgente.

Il confronto con il contratto Quando è disponibile la specifica OpenAPI, ogni segnalazione viene confrontata con il requisito di sicurezza, locale o globale, dell'operazione corrispondente. I casi possibili sono tre. Nel primo, il più interessante, il contratto dichiara l'operazione autenticata ma il codice non lega l'oggetto al chiamante. La regola di Spectral [35] dedicata ad API1 controlla che le operazioni su un identificativo dichiarino un requisito di sicurezza. Il livello statico copre esattamente questo punto cieco, e la confidenza della segnalazione cresce di 0,10 perché il codice smentisce una garanzia del contratto. Nel secondo caso anche il contratto è privo di protezione, e la segnalazione si aggiunge a quella di Spectral sulla stessa chiave. Nel terzo l'handler non compare affatto nel contratto: si tratta di un possibile endpoint non documentato, che il livello dinamico non attaccherebbe, perché costruisce il proprio inventario dalla specifica.

Il livello dinamico Il livello dinamico applica il *differential testing*: invia la stessa richiesta con identità diverse e ne confronta le risposte. Le identità sono tre: una *vittima* con ruolo ordinario, un *pari* con lo stesso ruolo e un'identità *privilegiata* con ruolo amministrativo, indicate in un file di configurazione; le richieste passano per OWASP ZAP [36], usato come proxy. Si provano gli endpoint che contengono almeno un identificativo, ricavati dalla pipeline, dalla specifica OpenAPI o dal traffico catturato. Le risorse di ciascuna identità sono create tramite endpoint di supporto esposti dal bersaglio, ripristinando lo stato dopo ogni richiesta che modifica i dati, oppure, se il bersaglio non collabora, ricavate dal traffico osservato, escludendo in tal caso i metodi che modificano i dati.

Per ogni endpoint e metodo il modulo prova tre scenari: nella BOLA *orizzontale* il pari tenta di accedere a una risorsa della vittima, nella BOLA *verticale* la vittima a una risorsa dell'identità privilegiata, e in un accesso di controllo l'amministratore accede alla risorsa della vittima, come il sistema dovrebbe consentire. Il confronto fra la risposta del proprietario, quella dell'attaccante e quella di una richiesta anonima stabilisce se l'accesso non autorizzato è riuscito; la segnalazione di runtime che ne deriva torna, attraverso la chiave comune, sulla corrispondente ipotesi statica, confermandola o smentendola.

L'analisi statica indica dove potrebbe trovarsi il difetto e raggiunge anche endpoint che il livello dinamico non vede. La verifica dinamica stabilisce se il difetto è reale e copre i controlli che sono presenti ma sbagliati.

3.4.2 API2 – L'identità del chiamante e la validità dei token

Nella *Broken Authentication* [6] il difetto riguarda la capacità del sistema di stabilire chi sia il chiamante. Dal codice il modulo ricostruisce un'idea del sistema di autenticazione, attraverso l'analisi delle librerie adottate, le funzioni di accesso, i middleware e i decoratori a protezione delle rotte e i claim inclusi nei token. Il codice non dice però come il meccanismo reagisce: se rifiuta un token falso o scaduto, se chiude la sessione dopo il logout, se frena i tentativi di accesso ripetuti. La divisione ricalca quella della letteratura: le debolezze che riguardano la firma e la validazione dei token dipendono da regole deterministiche e si prestano sia all'ispezione del codice sia alla generazione di token manipolati [37, 38], mentre l'assenza di freni sui tentativi di accesso emerge dall'osservazione del traffico più che da un'ispezione sintattica [7].

Il grafo di conoscenza dell'autenticazione Il modulo non presuppone di conoscere il bersaglio: ne ricostruisce la gestione dell'identità in un modello strutturato, il *grafo di conoscenza dell'autenticazione*, fondendo tre fonti. I file dichiarativi (manifest delle dipendenze, configurazione, infrastruttura) rivelano stack, librerie e protocolli delle credenziali; il codice sorgente le rotte di accesso, i middleware di protezione e, quando è cablato, il segreto usato per firmare i token, decisivo nella fase dinamica; la

specifica OpenAPI e l'eventuale traffico catturato gli schemi di sicurezza dichiarati e i claim, i ruoli e i permessi contenuti nei token. Il grafo porta con sé una confidenza fra 0 e 1 che cresce con ogni fonte concorde: è la correlazione fra livelli tradotta in un numero, e più avanti decide quali prove abbiano fondamento sufficiente per essere tentate.

La batteria di prove Ricostruito il modello, il modulo sollecita l'applicazione con richieste reali. Dopo un controllo di raggiungibilità e la localizzazione degli endpoint di autenticazione esegue una batteria di prove asincrone, ciascuna dedicata a una debolezza distinta e conclusa con un esito `PASS`, `FAIL` o `INCONCLUSIVE`, con severità e categoria interna. La Tabella 3.3 le raggruppa per l'aspetto dell'autenticazione che mettono alla prova.

Il legame più stretto con la parte statica passa per il segreto di firma. Quando è disponibile, la prova sul token scaduto non invia un token qualsiasi, ma ne costruisce uno crittograficamente valido con scadenza nel passato e osserva se il backend lo accetta, distinguendo così un difetto reale di validazione da un semplice rifiuto per firma non riconosciuta. Le prove che presuppongono una sessione autenticata vengono tentate solo se la confidenza del modello supera una soglia predefinita, per non produrre esiti privi di fondamento quando la ricostruzione è incerta, e l'esecuzione resta entro guardrail non distruttivi: la prova di forza bruta, per esempio, viene omessa se il bersaglio è dichiarato di produzione. Se il bersaglio non espone un proprio endpoint di login, perché delega il riconoscimento a un provider esterno, le prove che richiederebbero un token non falliscono ma si dichiarano non concludenti, tenendo separata l'assenza di una debolezza dall'impossibilità di verificarla. Nel punteggio di robustezza ogni prova fallita sottrae una quota proporzionale al proprio peso e riceve un'indicazione correttiva come nella Sottosezione 3.3.2.

La regola statica S01 Proprio quando l'autenticazione è delegata a un provider esterno la verifica dinamica ha meno presa, perché senza un login del bersaglio i token non si possono mettere alla prova. Per questo caso esiste la regola S01, puramente statica, che individua una funzione che scompone il token nei tre segmenti separati da punti, decodifica il payload in Base64 e lo interpreta come JSON senza mai invocare

Aspetto	Prova	Debolezza sollecitata
Firma e validazione dei token	T01	token manipolato accettato
	T02	token scaduto accettato
	T07	confusione di algoritmo (RS256→HS256)
	T17	iniezione di header JWKS
Gestione di sessione e token	T05	token ancora valido dopo il logout
	T06	session fixation
	T12	refresh token riutilizzabile
	T13	refresh token non ruotato
	T14	refresh token a durata illimitata
	T15	refresh token usato in parallelo
Flussi e credenziali	T04	token riusato su un'altra audience
	T08	escalation di privilegio tramite i claim
	T18	flussi OAuth2/OIDC senza protezioni (state, PKCE)
	T19	credenziali non basate su token
Forza bruta e limitazione	T03	login senza limite ai tentativi
	T20	limite aggirato con lo spoofing dell'header IP
Autenticazione a più fattori e SSO	T21	MFA aggirabile od OTP debole
	T22	firma delle asserzioni SAML manomessa
Comportamento e riservatezza	T09	cookie senza flag di sicurezza
	T10	informazioni sensibili nei messaggi di errore
	T11	enumerazione degli utenti
	T16	reset della password debole

Tabella 3.3: Le prove dinamiche del modulo, una per riga e raggruppate per aspetto dell'autenticazione.

una libreria di verifica della firma. È un ripiego insicuro tipico: in mancanza della validazione crittografica il codice si fida comunque del contenuto del token, e un attaccante può forgiarne uno con qualsiasi identità. La lettura non sicura del payload resta visibile nel sorgente anche quando il sistema non è raggiungibile o non offre un punto d'ingresso da provare: la componente statica della categoria rimane rilevabile proprio dove quella dinamica non lo è.

3.4.3 API3 – Le proprietà protette di un oggetto

La *Broken Object Property Level Authorization* [39] (Sottosezione 2.2.3) ha due direzioni: una proprietà riservata restituita a chi non dovrebbe vederla e una proprietà interna accettata in scrittura da chi non dovrebbe modificarla. La difficoltà propria di questa categoria sta a monte: escluso dal criterio un elenco di campi sensibili, il modulo deve dedurre da sé quali proprietà siano protette prima di cercare dove possano sfuggire.

Inventario e proprietà protette L'inventario degli oggetti e delle loro proprietà nasce da due fonti. Dal codice, in Python, JavaScript, TypeScript e Go, il modulo estrae le classi che modellano i dati, come gli oggetti di trasporto e le entità di persistenza; dal contratto OpenAPI legge gli schemi di richiesta e risposta di ciascuna operazione. Per confrontare le due fonti, i nomi vengono ricondotti a una forma comune: le classi `UserDTO` o `UserEntity` nel codice e il percorso `/users` nel contratto diventano entrambi `User`.

Una proprietà è considerata protetta quando l'applicazione stessa la tratta come tale. Il modulo cerca nel codice i punti che solo un utente con un certo ruolo può raggiungere, come una funzione riservata all'amministratore o il ramo di una condizione sul ruolo del chiamante, e considera protette le proprietà lette o modificate lì. Il riconoscimento non si basa sui nomi delle proprietà ma sui modi in cui i framework più diffusi dichiarano un controllo di ruolo, raccolti in un catalogo di configurazione estendibile. Il contratto aggiunge un secondo indizio, dotato di una direzione: un campo di sola lettura è protetto in scrittura, uno di sola scrittura, come una password, in lettura. Il modulo conserva questa direzione, per cui un campo di sola lettura che compare in una risposta non è un difetto.

Regole statiche e verifica per proprietà Individuate le proprietà protette, quattro regole statiche cercano i punti in cui la protezione può essere aggirata. La Tabella 3.4 le riassume indicando anche la direzione, lettura o scrittura, in cui ciascuna mette alla prova la protezione. La regola S01 cerca il *mass assignment*, cioè il corpo della richiesta copiato per intero su un oggetto, per esempio con un ciclo su tutti i campi ricevuti. Se il codice accetta solo i campi di un'*allowlist* la segnalazione cade; se ne esclude

alcuni con una denylist resta, perché ogni altro campo rimane scrivibile. La regola S02 cerca il caso opposto, un oggetto con proprietà protette serializzato e restituito così com'è fuori da un controllo di ruolo. Le regole S03 e S04 guardano il contratto: la prima segnala un campo di sola lettura che un'altra operazione accetta in scrittura, o di sola scrittura che compare in una risposta; la seconda una proprietà protetta che circola nel traffico ma che il contratto non documenta.

Regola	Direzione	Cosa segnala
S01	scrittura	corpo della richiesta copiato per intero su un oggetto, senza allowlist dei campi
S02	lettura	oggetto con proprietà protette restituito per intero, fuori da un controllo di ruolo
S03	entrambe	campo di sola lettura accettato in scrittura, o di sola scrittura restituito
S04	entrambe	proprietà protetta presente nel traffico ma assente dal contratto

Tabella 3.4: Le quattro regole statiche del modulo. Ogni segnalazione nasce "da verificare" e viene confermata o smentita dalle prove dinamiche della stessa direzione.

La verifica dinamica usa le credenziali di un utente ordinario e di un amministratore e agisce sempre sulla risorsa dell'utente ordinario, per esempio il suo profilo: l'accesso all'oggetto è quindi legittimo, e l'unico controllo messo alla prova è quello sulle proprietà. Le due direzioni vanno provate separatamente, perché un'API può accettare in scrittura una proprietà che non restituisce mai, oppure restituire dati riservati senza consentirne la modifica. Le prove sono sette. In lettura verificano se l'utente ordinario riceve una proprietà protetta e se la riceve esattamente come l'amministratore. In scrittura verificano se riesce a modificare una proprietà protetta, ad aggiungere alla richiesta campi che il client normalmente non invia (mass assignment) o proprietà non documentate dal contratto, a modificare una proprietà che non viene mai restituita e a far prevalere la propria scrittura su quella dell'amministratore.

Una prova di lettura si giudica dalla risposta, una di scrittura no, perché molti server rispondono con successo e scartano in silenzio i campi non ammessi: il modulo scrive quindi un valore riconoscibile, rilegge la risorsa come amministratore e conferma il difetto solo se quel valore risulta salvato. Gli esiti tornano poi sulle segnalazioni statiche, che diventano *confermate* se una prova della stessa direzione ha successo su una delle loro proprietà e *non confermate* se la protezione ha retto. Nel

punteggio di robustezza ogni tipo di prova riuscita sottrae la stessa quota.

3.4.4 API5 – Il ruolo richiesto per invocare una funzione

La *Broken Function Level Authorization* [19] (Sottosezione 2.2.5) riguarda le funzioni che un ruolo non dovrebbe poter invocare, come esportare l'elenco degli utenti, cancellare un account altrui o promuovere sé stessi ad amministratore. Rispetto alla BOLA la domanda non è più se un utente possa accedere a un certo oggetto, ma se possa eseguire una certa funzione. Il difetto nasce spesso da una dimenticanza: una rotta aggiunta a un controller eredita il routing e l'autenticazione, ma non l'autorizzazione, e resta raggiungibile da chiunque abbia effettuato l'accesso. Il rilevatore segue l'impianto statico a livelli, al quale aggiunge un livello dedicato ai file di configurazione e una regola che segue il flusso dei dati.

Funzioni privilegiate e controlli di ruolo Sulla mappa degli endpoint, un endpoint è considerato privilegiato quando il suo percorso corrisponde a uno schema tipico delle funzioni riservate, come `/admin`, `/internal`, `/debug`, `/config`, `/export` o `/users/{id}/role`, oppure quando è una cancellazione o una sostituzione rivolta a un'intera collezione anziché a un singolo elemento. Per i controlli, il modulo tiene distinti i decoratori di autenticazione, come `login_required`, da quelli di ruolo, come `require_role("admin")`, e riconosce questi ultimi anche quando sono scritti nel corpo della funzione, come un confronto sul ruolo dell'utente corrente. Su questa mappa operano le regole riepilogate nella Tabella 3.5. Come mostra la tabella, quasi tutte cercano una funzione privilegiata a cui manca il controllo di ruolo; fanno eccezione BF-005, che guarda la configurazione, e BF-007, che segue il ruolo inviato dal chiamante. La severità più alta spetta proprio ai due casi in cui chiunque può ottenere privilegi: la funzione riservata senza alcun controllo di ruolo (BF-001) e il ruolo scelto dal chiamante (BF-007).

Configurazione e contratto Nei file di configurazione il modulo cerca il *diniego per impostazione predefinita*, che il capitolo precedente ha indicato come difesa centrale della categoria. La regola BF-005 segnala le applicazioni in cui una rotta priva di una protezione esplicita resta aperta a tutti, anziché essere negata: è il caso, per esempio,

di un progetto che non imposta un permesso predefinito per tutte le rotte, o che applica l'autenticazione solo dopo averle registrate.

Nel contratto il modulo segnala le operazioni privilegiate prive di uno schema di sicurezza e i percorsi amministrativi esposti. Il confronto con il codice è particolarmente utile per BF-006: un endpoint di debug o di test presente nel codice e assente dal contratto è il caso più affidabile, perché l'assenza dal contratto conferma che quella funzione non doveva essere pubblica.

Regola	CWE	Debolezza ricercata	Severità
BF-001	285 [20]	endpoint privilegiato privo di controllo di ruolo	`CRITICAL`
BF-002	862 [40]	operazione sensibile (cancellazione, cambio di ruolo, rimborso) protetta solo dall'autenticazione	`HIGH`
BF-003	650 [41]	rotta privilegiata raggiungibile con qualsiasi verbo HTTP	`HIGH`
BF-004	284 [42]	rotte `/admin` senza un controllo di ruolo comune, o dati restituiti in blocco su un percorso ordinario	`HIGH`
BF-005	276 [43]	permesso predefinito anziché diniego nella configurazione	`MEDIUM`
BF-006	285 [20]	funzione di debug o di test rimasta esposta	`HIGH–LOW`
BF-007	269 [44]	ruolo scelto dal chiamante nel corpo della richiesta	`CRITICAL`

Tabella 3.5: Le sette regole del modulo. BF-005 opera sui file di configurazione; BF-001, BF-003, BF-004 e BF-006 sul codice e sul contratto OpenAPI; BF-002 e BF-007 sul solo codice. La severità di BF-006 dipende da quanto l'endpoint è protetto.

L'escalation di privilegio tramite il corpo della richiesta Alcuni difetti non si vedono in un singolo punto, come un decoratore, un percorso o una voce di configurazione, ma nel percorso che un dato compie all'interno della funzione. Il caso tipico è una funzione che crea un invito o aggiorna un profilo e prende il ruolo da assegnare direttamente dal corpo della richiesta: basta inviare `{"role": "admin"}` per ottenere privilegi che nessuno ha concesso. La regola BF-007 copre questo scenario seguendo il flusso dei dati all'interno della singola funzione: parte dal contenuto della richiesta e segue i campi che esprimono un privilegio, come `role`, `is_admin` o `permissions`, anche quando passano per variabili intermedie.

La regola scatta quando uno di questi valori viene scritto nel record dell'utente o salvato nel database. Per contenere i falsi positivi sono esclusi gli usi in sola lettura, perché un ruolo passato a un filtro o a una ricerca serve a selezionare dati e non ad assegnarli, e la segnalazione viene soppressa se la funzione verifica il privilegio del

chiamante prima dell'assegnazione. Infine, lo stesso endpoint rilevato sia nel codice sia nel contratto confluisce in una sola segnalazione.

3.5 Categorie tecniche: un metodo per categoria

Le categorie raccolte in questa sezione sono quattro: l'Unrestricted Resource Consumption (API4), la Server-Side Request Forgery (API7), la Security Misconfiguration (API8) e l'Unsafe Consumption of APIs (API10). Secondo il criterio della Sottosezione 3.1.1 si comportano come categorie tecniche: la violazione si riconosce leggendo l'elaborato di progetto, senza sollecitare il sistema in esecuzione, e i rilevatori sono statici.

Non condividendo un modello uniforme, ciascuna categoria richiede una logica di analisi dedicata e viene pertanto descritta separatamente.

3.5.1 API4 – I limiti sul consumo di risorse, su tre livelli

Ogni richiesta a un'API ha un costo in tempo di calcolo, memoria e banda, e in alcuni casi un costo economico diretto, come quando il server invia un SMS o interroga un modello di intelligenza artificiale a pagamento. L'*Unrestricted Resource Consumption* [11] (Sottosezione 2.2.4) riguarda le API che non pongono un limite a questo costo: una richiesta che chiede un numero arbitrario di risultati, un file di dimensioni eccessive o una sequenza di invii ripetuti basta a degradare il servizio o a generare costi non previsti.

Lo stesso limite può però essere definito in punti diversi del sistema: un controllo sulla dimensione dei file può trovarsi nel codice dell'applicazione, essere imposto dal proxy che la precede o essere dichiarato nel contratto dell'API. La sua assenza dal codice non basta quindi a stabilire la vulnerabilità. Il modulo analizza staticamente tutti e tre i livelli e ne confronta i risultati, come richiesto dal secondo requisito: qui la correlazione non è un raffinamento dell'analisi, ma la condizione per distinguere un'assenza locale da una reale assenza del limite nel sistema.

I tre livelli di analisi Su ciascun livello il modulo cerca i punti in cui un limite servirebbe e manca. La Tabella 3.6 elenca queste condizioni raggruppate per livello e mostra che lo stesso limite, come la dimensione di un caricamento o il numero di risultati di una paginazione, può comparire su più livelli: è questo che rende possibile il confronto. Nel codice sorgente, Python e JavaScript, si tratta di paginazioni, caricamenti, chiamate verso altri servizi, cicli su dati inviati dal client e, per le API GraphQL, della profondità di annidamento delle query. Nella configurazione contano i limiti di memoria dei container, la dimensione massima delle richieste e i timeout dei server; un timeout impostato a zero, che di fatto disattiva il controllo, ha severità maggiore di uno assente. Nel contratto OpenAPI, quando è disponibile, il modulo verifica i vincoli dichiarati sui parametri[1] e la protezione degli endpoint costosi.

Livello	Condizione segnalata
codice	paginazione con numero di risultati scelto liberamente dal client
	file caricato senza controllo della dimensione
	chiamata verso un altro servizio priva di timeout
	ciclo su un elenco inviato dal client senza limite di lunghezza
	servizio a pagamento invocabile senza limiti
	query GraphQL senza limite alla profondità di annidamento
configurazione	container senza limite di memoria
	dimensione massima delle richieste non fissata dal proxy o dall'applicazione
	timeout del server assente o impostato a zero
contratto	parametro di paginazione senza valore massimo
	caricamento senza dimensione massima
	operazione *batch* senza numero massimo di elementi
	endpoint costoso privo di protezione

Tabella 3.6: Le condizioni rilevate dal modulo, per livello di analisi. Le segnalazioni dei diversi livelli vengono poi confrontate fra loro.

Stabilire quali endpoint siano costosi è la parte più delicata e resta un'euristica. Il nome è un indizio, perché percorsi come `/sms` o `/export` suggeriscono un'elabo-

[1] Nel contratto OpenAPI i vincoli `maximum`, `maxLength` e `maxItems` dichiarano rispettivamente il valore, la lunghezza e il numero di elementi massimi ammessi per un campo.

razione onerosa, ma da solo può ingannare: leggere una fattura già emessa non ha costo, emetterla sì. Il modulo considera perciò anche ciò che l'operazione riceve: un numero di telefono accompagnato da un testo, un importo con la relativa valuta o un *prompt* destinato a un modello di intelligenza artificiale sono indizi più affidabili del solo nome. Il codice può inoltre chiarire un dubbio che il contratto lascia aperto: una rotta che invoca un servizio a pagamento è costosa anche quando ha un nome innocuo, e viene segnalata se il contratto non prevede alcuna protezione.

L'ultimo passo confronta i tre livelli. Il modulo risale da ogni segnalazione nel codice alla rotta che la contiene e la accosta all'operazione corrispondente del contratto. Quando lo stesso limite manca in entrambi, come un parametro `limit` senza tetto nel codice e nel contratto, le due segnalazioni vengono collegate e la confidenza aumenta. Quando invece un altro livello compensa la mancanza, per esempio un proxy che rifiuta i file troppo grandi, la segnalazione resta nel rapporto con severità `INFO` e con il riferimento alla difesa che la compensa.

3.5.2 API7 – Dall'input dell'utente alla chiamata di rete

La *Server-Side Request Forgery* [24] (Sottosezione 2.2.7) si ha quando un endpoint usa un dato fornito dall'utente per decidere verso quale indirizzo il server debba inviare una richiesta, senza controllare la destinazione. Un servizio che scarica l'immagine di profilo da un URL indicato dall'utente, per esempio, può essere indotto a interrogare indirizzi della rete interna, che dall'esterno non sarebbero raggiungibili. Il caso più grave riguarda il servizio di metadati delle macchine cloud, un indirizzo interno da cui ogni istanza legge la propria configurazione e, spesso, le proprie credenziali temporanee: chi riesce a farlo interrogare dal server può impossessarsene.

Il modulo è statico, e le fonti primarie sostengono questa scelta. OWASP giudica facile la rilevabilità della categoria, con la sola eccezione della forma cieca, in cui il server invia la richiesta ma non ne restituisce la risposta [24]; la CWE-918 indica come metodo più efficace l'analisi statica automatica [25]. Il motivo è che qui la domanda è una proprietà del codice: un dato dell'utente arriva o non arriva a una chiamata di rete, e lo si può stabilire senza conoscere le intenzioni di chi ha scritto il programma. Gli studi sull'argomento ne mostrano anche i limiti, dalla necessità di verificare a

mano i risultati [45] ai percorsi segnalati ma non sfruttabili [46], fino al ricorso a tecniche dinamiche [47]. Per il modulo ne derivano tre esigenze: riconoscere tutte le funzioni con cui il codice invia richieste di rete, segnalare l'input dell'utente solo quando decide la destinazione della richiesta, e accorgersi di quando l'indirizzo viene controllato prima dell'uso.

Il flusso dei dati Il modulo usa Semgrep, uno strumento che cerca nel codice schemi descritti da regole, per seguire il percorso dell'input dell'utente all'interno del programma (*analisi della contaminazione*). Il percorso parte dai punti in cui l'input entra nell'applicazione, come i parametri della query o il corpo della richiesta, e arriva alle funzioni che inviano richieste ad altri server, nelle librerie più diffuse di Python e di JavaScript. Non basta però che l'input arrivi alla chiamata: la segnalazione scatta solo se finisce nell'indirizzo di destinazione, e non per esempio nel contenuto inviato. Basta anche che l'utente controlli una parte dell'indirizzo, come quando il suo input viene aggiunto in coda a un URL fisso, perché anche così può dirottare la richiesta verso una risorsa diversa da quella prevista.

Il percorso si considera interrotto, e la chiamata non viene segnalata, quando lungo la strada l'indirizzo viene verificato, per esempio confrontandolo con un elenco di destinazioni ammesse. La Tabella 3.7 riepiloga le regole del modulo: le prime tre applicano l'analisi appena descritta alle diverse librerie di rete, SS-004 e SS-005 cercano due situazioni specifiche, descritte di seguito, e SS-006 lavora sul contratto.

Regola	Cosa segnala	Livello
SS-001–003	input dell'utente nell'indirizzo di una richiesta di rete (una regola per ciascun gruppo di librerie, in Python e in Node.js)	codice
SS-004	richiesta verso un indirizzo dell'utente che segue in automatico i reindirizzamenti	codice
SS-005	chiamata verso l'indirizzo del servizio di metadati cloud	codice
SS-006	parametro che rappresenta un URL senza vincoli sul formato o sulle destinazioni ammesse	contratto

Tabella 3.7: Le sei regole del modulo: cinque sul codice sorgente e una sul contratto OpenAPI.

Due regole vanno oltre il percorso dell'input. SS-004 segnala le chiamate che accettano di essere rimandate a un altro indirizzo: un server esterno autorizzato

potrebbe infatti reindirizzare la richiesta verso la rete interna dopo che il controllo sull'indirizzo è già stato superato. SS-005 cerca invece direttamente nel codice l'indirizzo del servizio di metadati cloud, che un'applicazione non ha in genere motivo di interrogare.

Il contratto SS-006 lavora sulla specifica OpenAPI. Cerca i campi in cui il client indica un indirizzo, come `webhook` o `callback_url`, e verifica che il contratto ne limiti i valori ammessi. Il contratto non dice se l'implementazione sia vulnerabile, ma mostra se l'interfaccia lascia aperta la porta già in fase di progetto.

Resta fuori dalla portata del modulo la forma cieca, l'unica che OWASP considera difficile da rilevare: poiché la risposta non torna all'attaccante, per accorgersene bisogna osservare le richieste che il server invia davvero, e quindi un sistema in esecuzione.

3.5.3 API8 – La configurazione confrontata con i valori di riferimento

La *Security Misconfiguration* [21] (Sottosezione 2.2.8) raccoglie errori di configurazione che possono verificarsi a diversi livelli, dalla rete fino all'applicazione. Questa sottosezione ne affronta la parte visibile nel codice sorgente e nei file di progetto, completando l'analisi infrastrutturale già trattata nella Sezione 3.3. Il modulo adotta un approccio statico: cerca direttamente nel codice le scelte di configurazione insicure, senza richiedere l'avvio o l'esecuzione del software.

Qui il rilevamento statico parte avvantaggiato. Nelle categorie di autorizzazione uno strumento non sa se un controllo fosse dovuto, e per il consumo di risorse non sa se un limite sia applicato altrove. Per la configurazione, invece, esiste un riferimento pubblico: valori raccomandati, impostazioni predefinite note per essere insicure, versioni di librerie con vulnerabilità note. Basta confrontare il progetto con quel riferimento, senza dover dedurre le intenzioni di chi lo ha scritto. Per questo OWASP giudica facile la rilevabilità della categoria e rinvia agli strumenti automatici [21].

Regole locali e regole globali Il modulo analizza la struttura sintattica dei file Python ed esegue controlli testuali su quelli JavaScript e TypeScript, escludendo le directory non applicative (come ambienti virtuali o cartelle di build). I controlli sono articolati in verifiche locali su ciascun file e verifiche globali sul progetto. Come mostra la Tabella 3.8, sei regole esaminano un singolo file, mentre tre valutano il progetto nel suo insieme: l'assenza degli header di sicurezza o di una politica CORS e le dipendenze in versioni vulnerabili. La severità più alta spetta al segreto scritto nel codice (SC-005), che chiunque legga il repository può riutilizzare. Ogni segnalazione porta con sé un grado di confidenza, e quelle sotto la soglia di 0,70 vengono scartate prima di comporre il rapporto.

Regola	Comportamento ricercato	Ambito	Severità
SC-001	politica CORS con carattere jolly o aperta assieme alle credenziali	file	`HIGH`
SC-002	modalità di debug attiva	file	`HIGH`
SC-003	gestore d'errore che espone la traccia dello stack	file	`HIGH`
SC-004	header di sicurezza mancanti (come HSTS e `Content-Security-Policy`)	globale	`MEDIUM`
SC-005	segreto cablato nel codice (entropia di Shannon)	file	`CRITICAL`
SC-006	assenza totale di politica CORS applicativa	globale	`MEDIUM`
SC-007	dipendenze in una versione presente in un elenco di versioni vulnerabili	globale	`HIGH`
SC-008	metodi HTTP non ristretti o operazioni che modificano dati esposte tramite GET	file	`MEDIUM`
SC-009	assenza di `Cache-Control` su rotte con dati sensibili	file	`MEDIUM`

Tabella 3.8: Il catalogo delle regole del modulo di rilevamento applicativo per API8:2023.

3.5.4 API10 – La fiducia nelle risposte dei servizi esterni

Nella maggior parte delle categorie viste finora il pericolo arriva dai dati che l'API riceve dai suoi utenti. L'*Unsafe Consumption of APIs* [29] (Sottosezione 2.2.10) ribalta il punto di vista: qui il pericolo arriva dai servizi di terze parti da cui l'applicazione stessa dipende, come un gateway di pagamento o un fornitore di dati esterni. Avendoli scelti e configurati, l'applicazione tende a fidarsi delle loro risposte senza troppe domande; ma quelle risposte attraversano pur sempre un confine di fiducia che spesso nessuno sorveglia. OWASP elenca cinque forme di questa fiducia mal

riposta: canale non cifrato, dati non validati, redirezioni seguite ciecamente, risorse senza limiti e timeout assenti. Nel repository del progetto OWASP è stato segnalato che alcune di queste forme si sovrappongono ad altre categorie della stessa lista [48].

Il modulo si occupa di tre forme (canale in chiaro, dati non validati e redirezioni cieche) e lascia le altre due, l'assenza di limiti e di timeout, al modulo dell'Unrestricted Resource Consumption (Sottosezione 3.5.1), che ne è la sede naturale; così lo stesso difetto non viene contato due volte, come chiede il criterio della Sottosezione 3.1.1. Queste tre forme si riconoscono direttamente nel codice, e per questa parte la categoria si comporta come una categoria *tecnica*: il rilevatore è statico e lavora sui sorgenti Python. Resta fuori ciò che il codice non può dire, cioè se la fiducia mal riposta porti davvero a una compromissione: è un residuo di natura *dinamica* nel senso del criterio, ripreso nel Capitolo 4.

Le tre regole Il modulo percorre i file Python del bersaglio, li legge con l'analizzatore sintattico della libreria standard e applica tre regole. La Tabella 3.9 le riassume: ciascuna corrisponde a una CWE distinta e a una delle tre forme di fiducia mal riposta coperte dal modulo, e tutte hanno severità alta.

Regola	CWE	Comportamento ricercato	Severità
UC-001	20 [49]	dati di terze parti usati senza validazione	`HIGH`
UC-002	319 [50]	consumo di un servizio esterno su canale in chiaro (`http`)	`HIGH`
UC-003	601 [51]	redirezioni di terze parti seguite ciecamente	`HIGH`

Tabella 3.9: Le tre regole del modulo, corrispondenti alle condizioni di consumo insicuro non già coperte dalla Sottosezione 3.5.1.

Le tre regole cercano segnali di natura diversa. La prima segue un dato. Di solito l'analisi del flusso dei dati parte dall'input dell'utente, mentre qui il dato da sorvegliare è la risposta di un servizio esterno: la regola prende quindi come punto di partenza il contenuto restituito da una chiamata HTTP in uscita, per esempio il risultato di `requests.get(...).json()`, e ne segue l'uso fino al punto in cui viene salvato o passato ad altri componenti senza controlli. È il caso dell'iniezione differita: un contenuto malevolo entra dal canale, ritenuto fidato, di una terza parte e fa danni più tardi, quando l'applicazione lo rilegge e lo usa. La seconda guarda un valore: l'indirizzo del servizio chiamato, che non deve cominciare con `http://`. La

terza guarda un'opzione che manca: `requests`, per esempio, segue le redirezioni in automatico a meno che la chiamata non contenga `allow_redirects=False`, e la regola segnala le chiamate verso terze parti in cui quell'opzione è assente.

Vale la stessa soglia di confidenza del modulo precedente, e il rapporto indica, per ciascuna regola, se ha prodotto almeno una segnalazione e riassume i risultati per severità e categoria.

L'insieme dei moduli descritti completa così la strategia di rilevamento del capitolo, fondata sulla combinazione di analisi statica multi-layer (codice, contratti OpenAPI e definizioni IaC) e verifica dinamica mirata per le categorie ibride.

CAPITOLO 4

Validazione preliminare dello strumento

Questo capitolo riporta la validazione dello strumento presentato nel Capitolo 3 e stabilisce in che misura l'obiettivo della tesi sia stato raggiunto. Dopo aver presentato le domande di ricerca e le evidenze raccolte, il capitolo riporta i risultati, li discute per categoria alla luce della classificazione, risponde alle domande di ricerca e si chiude con i limiti dello strumento.

4.1 Obiettivi e impostazione della validazione

4.1.1 Domande di ricerca

Come discusso nel Capitolo 2, le categorie dell'OWASP API Security Top 10 (2023) differiscono per il luogo in cui la violazione lascia traccia: alcune sono vulnerabilità *tecniche*, riconoscibili negli artefatti di progetto, altre dipendono dalla logica di business. Lo studio applica questa distinzione alle dieci categorie e ne verifica le conseguenze con lo strumento descritto nel Capitolo 3. Su questa base si formula la domanda di ricerca principale.

> **RQ$_1$:** *Quali categorie dell'OWASP API Security Top 10 (2023) si possono rilevare con l'analisi statica degli artefatti di progetto, e quali richiedono un'evidenza che questi non contengono?*

La domanda si scompone in due interrogativi subordinati.

> **RQ$_{1.1}$:** *Per le categorie rilevabili staticamente, quali livelli di analisi occorre combinare per coprirle?*

> **RQ$_{1.2}$:** *Per le categorie non rilevabili interamente con l'analisi statica, quale parte resta riconoscibile staticamente e di che natura è ciò che manca?*

Ciò che manca, il *residuo*, può essere accertabile sul sistema in esecuzione, recuperabile estendendo l'analisi ad altri file del progetto, esplicitabile come parametro dell'analisi, oppure legato alla sola conoscenza del dominio applicativo (Sottosezione 3.1.1).

Lo studio ha forma costruttiva: per ogni categoria tecnica o ibrida è stato realizzato un modulo che verifica la condizione individuata dal criterio, e lo si mette alla prova. Lo strumento è quindi il mezzo con cui la classificazione viene resa operativa, non l'oggetto della ricerca: se un modulo non rileva un difetto, la causa può stare nei limiti dell'implementazione e non nel criterio.

4.1.2 Le evidenze raccolte

Le evidenze sono di tre tipi e vanno lette in modo diverso.

1. **Casi di prova dedicati:** per ogni modulo una coppia di applicazioni gemelle, una vulnerabile e una irrobustita, in Flask o Express, e per l'infrastruttura un insieme di manifest Terraform. Dicono se ogni modulo riconosce la condizione per cui è stato costruito senza segnalare la versione corretta.

2. **Codice non scritto per i moduli:** repository pubblici e il benchmark OWASP crAPI [52]. Mostrano come si comporta lo strumento quando la condizione non è stata preparata per lui.

3. **Insieme di prova:** tre applicazioni di terzi riservate alla validazione e analizzate con tutti i moduli in un'unica campagna: sadcloud [53] per l'infrastruttura Terraform, VAmPI [54] e DVAPI [55] per le API.

Una segnalazione corrisponde a una vulnerabilità documentata quando riguarda la stessa categoria OWASP e lo stesso endpoint, identificato da metodo e percorso, qualunque sia il livello di analisi che l'ha prodotta.

4.2 Risultati

4.2.1 Casi di prova

Sui manifest Terraform il modello di rischio ordina le risorse come atteso: le cinque configurazioni aperte o prive di autorizzazione (ACL pubblica, chiave cablata in una Lambda, policy IAM $*$ su $*$, metodi senza autorizzazione, blocchi di accesso disattivati) ottengono punteggi fra 6,8 e 9,2, mentre i bucket protetti si fermano a 3,2, un punteggio dovuto ai soli irrobustimenti.

La Tabella 4.1 riassume gli esiti sulle applicazioni gemelle. Tutti i moduli rilevano le debolezze introdotte senza falsi positivi sulle versioni irrobustite. Nella BOPLA si vede inoltre la cooperazione fra i livelli: la segnalazione S04, vera sul contratto, è smentita dal sistema in esecuzione. Per la Broken Authentication, le cui prove sono dinamiche, cinque prove falliscono sulla versione vulnerabile e nessuna su quella corretta.

Questi esiti dicono che ogni modulo fa ciò per cui è stato costruito; la sua efficacia reale emerge solo su codice non scritto ad hoc.

4.2.2 Codice non scritto per i moduli

Repository esterni Su un repository con vulnerabilità etichettate, il livello statico della BOLA segnala tutti e quattro i metodi della risorsa vulnerabile e nessuno di quella protetta; uno dei quattro non compare nella specifica OpenAPI, e quindi non sarebbe stato raggiunto da un'analisi solo dinamica. Il livello dinamico conferma tre dei quattro metodi (GET, PUT e PATCH, non la cancellazione) e non segnala la risorsa

Modulo	Versione vulnerabile	Versione corretta	VP	FP
BOLA (livello statico)	12 casi su 12 segnalati	nessuna segnalazione	12	0
Broken Authentication	5 prove su 22 falliscono	nessuna prova fallisce	–	–
BOPLA	S01, S02 e S04 segnalate e confermate a runtime	resta S04, vera sul contratto e smentita a runtime	3	0
Resource Consumption	9 casi su 9 segnalati	nessuna segnalazione	9	0
BFLA	7 debolezze su 7 segnalate	nessuna segnalazione	7	0
SSRF	3 casi su 3 segnalati	nessuna segnalazione	3	0
Security Misconfiguration	7 casi su 7 segnalati	nessuna segnalazione	7	0
Unsafe Consumption	3 regole su 3 segnalate	nessuna segnalazione	3	0

Tabella 4.1: Esiti sulle coppie gemelle. VP e FP indicano i veri e i falsi positivi; per la Broken Authentication, le cui prove sono dinamiche, si riporta il numero di prove fallite.

protetta. Lo strumento segnala anche un endpoint di importazione che consente di cambiare il proprietario di un progetto: il caso non è fra quelli documentati né è stato raggiunto dalla prova dinamica, e resta quindi da verificare.

Su un secondo repository, accompagnato dalla specifica OpenAPI, la BFLA produce tre segnalazioni BF-006 su endpoint privi di autenticazione e di controllo del ruolo. Si tratta però di endpoint usati per preparare i dati delle prove automatiche, non di vulnerabilità; gli endpoint dell'applicazione non vengono segnalati.

crAPI Sulla parte infrastrutturale di crAPI le 240 segnalazioni iniziali si riducono a 46 voci dopo filtraggio e raggruppamento, con problemi reali e un solo falso positivo. Sulle API, invece, come mostra la Tabella 4.2, nessuna delle sfide documentate riconducibile alla Top 10 viene rilevata: i livelli statici non coprono il servizio Java e non riconoscono le forme di codice usate da crAPI, come le viste Django definite come classi o i campi assegnati uno per uno.

Sul bersaglio realistico usato per l'Unsafe Consumption le regole non producono segnalazioni, ma perché il difetto non c'è: gli indirizzi delle terze parti sono definiti solo a runtime e non esistono punti di iniezione non protetti. L'esito quindi non dice nulla sulla sensibilità del modulo.

Sfida di crAPI	Cat.	Esito	Motivo
1 – veicolo di un altro utente	API1	non segnalata	servizio Java, fuori dal livello statico; livello dinamico non eseguito su crAPI
2 – report dei meccanici altrui	API1	non segnalata	vista Django definita come classe, identificativo nella query
4, 5 – dati altrui e proprietà interne esposti	API3	non segnalate	nessuna segnalazione statica
8–10 – mass assignment su ordini, saldo e video	API3	non segnalate	campi assegnati uno per uno, fuori dalla regola S01
3, 14, 15 – reset della password, accesso senza autenticazione, JWT falsificabili	API2	non valutate	esiti del modulo non riportati
6 – DoS tramite “contact mechanic”	API4	non valutata	esiti del modulo non riportati
7 – cancellazione di un video altrui	API5	non valutata	modulo non eseguito su crAPI
11 – richiesta verso un host esterno	API7	non valutata	esiti del modulo non riportati
12, 13, 16–18 – iniezioni e chatbot	–	fuori perimetro	non appartengono alla Top 10 API

Tabella 4.2: Sfide documentate da crAPI alla revisione 1.1.5, categoria OWASP ed esito dello strumento.

4.2.3 Insieme di prova

La Tabella 4.3 riporta le segnalazioni di ogni modulo sui bersagli dell’insieme di prova. VAmPI compare due volte perché può essere avviata in modalità vulnerabile o sicura con lo stesso codice.

Il risultato più chiaro viene da VAmPI: l’analisi statica restituisce le stesse segnalazioni nelle due modalità, e solo il livello dinamico della BOLA le distingue. Nella modalità vulnerabile conferma che un utente può leggere il libro di un altro (`GET /books/v1/{titolo}`), la vulnerabilità documentata da VAmPI; nella modalità sicura le stesse richieste vengono respinte. La differenza sta nel comportamento, non nel codice, e nessuna analisi statica potrebbe vederla. Su DVAPI la BOLA dinamica e la Broken Authentication non sono state eseguite per limiti dello strumento (Sezione 4.5).

La Tabella 4.4 confronta le segnalazioni con le vulnerabilità documentate da

Modulo	VAmPI vuln.	VAmPI sicuro	DVAPI	sadcloud
Scansione statica (voci)	28	28	73	38
BOLA, livello dinamico	1 su 12 prove	0 su 12 prove	non eseguita	–
BOLA, livello statico	0	0	0	0
Broken Authentication	1	1	non eseguita	–
BOPLA	0	0	0	0
Resource Consumption	3	3	3	0
BFLA	0	0	1	0
SSRF	10	10	0	0
Security Misconfiguration	5	5	3	1
Unsafe Consumption	0	0	0	0

Tabella 4.3: Voci della scansione statica e segnalazioni per modulo sui bersagli dell'insieme di prova. Un trattino indica che il modulo non si applica: sadcloud non ha un'applicazione in esecuzione.

ciascun progetto. La *recall* è la quota di vulnerabilità trovate; la *recall nel perimetro* esclude quelle che nessun modulo può trovare per costruzione. La *precisione minima* è la quota di segnalazioni che corrispondono a una vulnerabilità documentata: è una stima per difetto, perché conta come errate anche le segnalazioni non esaminate a mano.

Bersaglio	Voci	VP	FN	Fuori perim.	Recall	Recall nel perim.	Prec. minima
VAmPI	8	2	3	3	25%	40%	11%
DVAPI	9	1	5	3	11%	17%	2%
sadcloud	84	51	33	0	61%	61%	25%

Tabella 4.4: Metriche di accuratezza sull'insieme di prova. Le voci fuori dalla Top 10 (una per VAmPI, una per DVAPI) sono escluse.

I risultati sono molto diversi fra infrastruttura e API. Su sadcloud Checkov trova 51 delle 84 misconfiguration documentate: le trova quando il problema riguarda una singola risorsa, come un database non cifrato o un bucket pubblico, e le perde quando riguarda l'account nel suo insieme, come l'assenza di CloudTrail. Sulle API lo strumento trova poco: delle tre corrispondenze, solo la BOLA di VAmPI confermata dal livello dinamico riconosce davvero la vulnerabilità, mentre le altre due coincidono con la voce documentata per categoria ed endpoint ma segnalano un problema diverso. Restano mancati, fra gli altri, il mass assignment di VAmPI e, su DVAPI, il caricamento di file senza limiti e la SSRF. Le precisioni minime sono basse

perché gran parte delle segnalazioni viene dalle regole sul contratto OpenAPI e, non essendo stata esaminata a mano, conta come errata.

4.3 Discussione per categoria

4.3.1 Categorie ibride

Le prove confermano la divisione del lavoro prevista dalla classificazione. La parte statica trova la condizione necessaria, cioè l'assenza del controllo, e raggiunge punti che la verifica dinamica non vedrebbe, come il metodo del repository esterno assente dalla specifica. La parte dinamica restituisce ciò che il codice non può dire: smentisce la segnalazione S04 della BOPLA, distingue la risorsa vulnerabile da quella protetta sul repository etichettato e, su VAmPI, separa due modalità che hanno lo stesso codice.

Nella BFLA il residuo resta aperto. Invocare una funzione riservata è facile, ma un `200 OK` attesta che l'operazione è stata eseguita, non che fosse legittimo eseguirla. La letteratura affronta il problema ricostruendo la specifica mancante dei ruoli [56, 57, 58] o scrivendo regole di sicurezza da verificare con fuzzing [59]. Le tre segnalazioni BF-006 sul repository esterno illustrano il punto: la regola stabilisce che quelle funzioni sono raggiungibili senza ruolo, non se debbano esserlo, e a deciderlo è chi conosce lo scopo degli endpoint.

Nella Broken Authentication le prove dinamiche distinguono la versione vulnerabile da quella irrobustita, ma possono accertare un difetto solo dove il meccanismo esiste e una sessione è disponibile.

4.3.2 Categorie tecniche

Per le quattro categorie tecniche le coppie gemelle danno esiti pieni, confermando che la condizione si legge negli artefatti. Resta però, per due di esse, una parte che solo il sistema in esecuzione può dire. Nell'Unrestricted Resource Consumption il modulo sa dire che un limite manca, ma non se la sua assenza basti a mettere in difficoltà il servizio: per questo servono prove di carico [11, 60]. Una segnalazione va letta quindi come una protezione da aggiungere. Nell'Unsafe Consumption l'analisi

statica individua la *predisposizione* (validazione assente, canale in chiaro, redirezione seguita ciecamente), mentre la sfruttabilità richiederebbe di mettere il sistema davanti a un fornitore ostile e controllato.

Sull'insieme di prova, invece, la Server-Side Request Forgery e la Security Misconfiguration non trovano le voci documentate di DVAPI: né la richiesta verso un URL arbitrario né la traccia dello stack restituita con un token non valido.

4.3.3 Categorie senza modulo

API6:2023 ricade fra le categorie di logica di business: nessun file del progetto dice se un flusso vada protetto dall'abuso, e il flusso viene percorso correttamente, con l'attacco che sta nel ripeterlo. Restano le contromisure comportamentali a runtime indicate da OWASP [23]. API9:2023 resta invece fuori dalla tripartizione, perché presuppone che il sistema da valutare non sia noto (Sottosezione 3.1.1): il confronto fra rotte del codice e contratto può suggerire API ombra o documentazione obsoleta, ma non dice nulla sugli host su cui l'applicazione è distribuita, che richiedono una ricognizione dell'infrastruttura.

4.4 Risposta alle domande di ricerca

Risposta a $\mathbf{RQ}_{1.1}$ Per le categorie tecniche servono, oltre al codice, il contratto OpenAPI e la configurazione di infrastruttura, e i segnali dei diversi livelli vanno correlati: un livello da solo non basta a dare un giudizio. L'esempio più chiaro è l'Unrestricted Resource Consumption, dove un limite assente dal codice diventa una segnalazione fondata solo se neanche proxy e contratto lo applicano. Sulla parte infrastrutturale il manifest va combinato con il catalogo delle policy, che distingue esposizioni e irrobustimenti, e con la correlazione, che riunisce i controlli sulla stessa risorsa: così su crAPI 240 segnalazioni si riducono a 46 voci. Sui casi di prova questa combinazione funziona per tutte e quattro le categorie tecniche, senza falsi positivi. Su codice di terzi funziona per l'infrastruttura (61% delle misconfiguration di sadcloud), ma non ancora per le API: nessuna delle vulnerabilità tecniche documentate da VAmPI e DVAPI viene trovata.

Risposta a $RQ_{1.2}$ Nelle categorie ibride l'analisi statica riconosce la condizione necessaria, cioè l'assenza del controllo, anche dove la verifica dinamica non arriva. Ciò che manca è di natura dinamica: stabilire se l'accesso viene davvero concesso richiede di sollecitare il sistema in esecuzione con identità diverse. VAmPI lo mostra nel modo più diretto: le due modalità hanno lo stesso codice, i livelli statici le trattano allo stesso modo e solo il livello dinamico della BOLA le distingue. Il residuo assume però forme diverse nelle quattro categorie:

- nella **BOLA** e nella **BOPLA** è accertabile a runtime: il livello dinamico conferma tre dei quattro metodi segnalati sul repository etichettato e smentisce la segnalazione S04 della BOPLA;
- nella **Broken Authentication** è accertabile solo a condizione che il meccanismo esista e una sessione sia disponibile;
- nella **BFLA** resta aperto: una risposta positiva non dice se l'invocazione fosse legittima, e decidere richiede di conoscere lo scopo della funzione, un'informazione che il criterio non fornisce all'analisi.

Risposta a RQ_1 Delle dieci categorie, quattro sono *tecniche* (API4, API7, API8 e API10) e si riconoscono sugli artefatti, come le misconfiguration dell'infrastruttura; quattro sono *ibride* (API1, API2, API3 e API5) e richiedono in aggiunta un'evidenza dinamica, ottenuta sollecitando il sistema con identità diverse; API6 è di *logica di business*; API9 resta fuori dalla classificazione perché presuppone un sistema non noto.

L'obiettivo della tesi è quindi raggiunto in parte. Lo è sul piano della classificazione: sulle coppie gemelle ogni modulo distingue la versione vulnerabile da quella corretta, e su VAmPI la differenza fra le due modalità emerge esattamente dove la classificazione la colloca, nel livello dinamico di una categoria ibrida. Non lo è ancora sul piano della generalità: su crAPI i livelli statici non segnalano nessuna delle sfide documentate, e sull'insieme di prova la recall nel perimetro è del 40% su VAmPI e del 17% su DVAPI, contro il 61% sull'infrastruttura di sadcloud. La classificazione indica fin dove lo strumento può arrivare; il divario fra quel limite e ciò che lo strumento raggiunge oggi su codice di terzi è il punto da cui partire per estenderlo.

4.5 Limiti

Presenza, non correttezza L'analisi statica accerta la presenza di un controllo, non la sua correttezza. Nella BOLA un filtro sul campo sbagliato viene accettato come legame, e una funzione chiamata `authorize` viene accettata senza esaminarne il contenuto; nella BFLA una riga che cita il ruolo senza bloccare l'esecuzione viene scambiata per una protezione.

Forme di codice e perimetro Identità, controlli e funzioni riservate sono riconosciuti attraverso nomi e forme d'uso comuni nei framework supportati: ciò che se ne discosta può sfuggire. Il livello statico della BOLA esamina un file alla volta, la BFLA non analizza le viste definite come classi e la BOPLA non analizza il codice Java. Il caso di crAPI mostra quanto questi confini pesino su un bersaglio realistico.

Configurazione Il manifest mostra ciò che dichiara, non ciò che è attivo, e il flag di esposizione dice che una risorsa è raggiungibile da chiunque, non se debba esserlo: un bucket che serve contenuti pubblici viene segnalato come uno che espone dati riservati.

Parte dinamica La parte dinamica richiede identità autenticate. Senza una sessione la Broken Authentication si riduce alla componente statica, e la BOPLA lascia le proprie segnalazioni da verificare. Il livello dinamico della BOLA richiede token con il claim `sub`: DVAPI, che usa `userId`, ne resta escluso. Sui bersagli esterni, inoltre, vengono provati solo i metodi che non modificano i dati.

Il prezzo del criterio Il criterio vieta di indicare all'analisi ciò che va protetto. Nella BOPLA una proprietà è quindi protetta solo se l'applicazione la tratta come tale in qualche punto, e un dato sensibile restituito a tutti senza alcun controllo non viene segnalato. È una conseguenza voluta: un elenco fisso di campi sensibili avrebbe funzionato sulle applicazioni di prova e su nessun altro sistema.

CAPITOLO 5

Conclusioni

Le conclusioni qui...

Bibliografia

[1] OWASP Foundation, "A05:2021 – security misconfiguration," https://owasp.org/Top10/2021/A05_2021-Security_Misconfiguration/, 2021, oWASP Top 10:2021. [Online]. Available: https://owasp.org/Top10/2021/A05_2021-Security_Misconfiguration/ (Citato alle pagine 4 e 5)

[2] SentinelOne, "Cloud misconfigurations: Risks, examples & prevention," https://www.sentinelone.com/cybersecurity-101/cloud-security/cloud-misconfigurations/, 2025. [Online]. Available: https://www.sentinelone.com/cybersecurity-101/cloud-security/cloud-misconfigurations/ (Citato a pagina 5)

[3] Cloud Computing News, "10 real-life cloud security failures and what we can learn from them," https://www.cloudcomputing-news.net/news/10-real-life-cloud-security-failures-and-what-we-can-learn-from-them/, 2025. [Online]. Available: https://www.cloudcomputing-news.net/news/10-real-life-cloud-security-failures-and-what-we-can-learn-from-them/ (Citato a pagina 5)

[4] OWASP Foundation, "OWASP API security top 10 – 2023," https://owasp.org/API-Security/editions/2023/en/0x11-t10/, 2023. [Online]. Available: https://owasp.org/API-Security/editions/2023/en/0x11-t10/ (Citato alle pagine 6, 15 e 32)

[5] OWASP API Security Project, "API1:2023 broken object level authorization," https://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization/, 2023, oWASP API Security Top 10 – 2023. Consultato il 25 agosto 2026. [Online]. Available: https://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization/ (Citato alle pagine 8, 19 e 40)

[6] ——, "API2:2023 Broken Authentication," https://owasp.org/API-Security/editions/2023/en/0xa2-broken-authentication/, 2023, consultato il 25 agosto 2026. [Online]. Available: https://owasp.org/API-Security/editions/2023/en/0xa2-broken-authentication/ (Citato alle pagine 14, 15, 19 e 43)

[7] OWASP Foundation, "Credential Stuffing," https://owasp.org/www-community/attacks/Credential_stuffing, oWASP Community Pages. [Online]. Available: https://owasp.org/www-community/attacks/Credential_stuffing (Citato alle pagine 14 e 43)

[8] Y. Sheffer, D. Hardt, and M. Jones, "JSON Web Token Best Current Practices," RFC 8725, Internet Engineering Task Force (IETF), BCP 225, feb 2020. (Citato a pagina 15)

[9] D. Temoshok, D. Proud-Madruga, Y.-Y. Choong, R. Galluzzo, S. Gupta, C. LaSalle, N. Lefkovitz, and A. Regenscheid, "Digital identity guidelines: Authentication and authenticator management," National Institute of Standards and Technology, NIST Special Publication 800-63B-4, jul 2025. (Citato alle pagine 15 e 19)

[10] OWASP Foundation, "Authentication Cheat Sheet," https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html, oWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html (Citato a pagina 15)

[11] OWASP API Security Project, "API4:2023 Unrestricted Resource Consumption," https://owasp.org/API-Security/editions/2023/en/0xa4-unrestricted-resource-consumption/, 2023, consultato il 25 agosto

2026. [Online]. Available: https://owasp.org/API-Security/editions/2023/en/0xa4-unrestricted-resource-consumption/ (Citato alle pagine 18, 19, 50 e 64)

[12] A. Cha, E. Wittern, G. Baudart, J. C. Davis, L. Mandel, and J. A. Laredo, "A principled approach to GraphQL query cost analysis," in *Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2020)*. ACM, 2020, pp. 257–268. (Citato alle pagine 18 e 19)

[13] S. A. Crosby and D. S. Wallach, "Denial of service via algorithmic complexity attacks," in *Proceedings of the 12th USENIX Security Symposium*. Washington, DC: USENIX Association, 2003, pp. 29–44. (Citato a pagina 18)

[14] C.-A. Staicu and M. Pradel, "Freezing the web: A study of ReDoS vulnerabilities in JavaScript-based web servers," in *Proceedings of the 27th USENIX Security Symposium (USENIX Security 18)*. Baltimore, MD: USENIX Association, 2018, pp. 361–376. (Citato alle pagine 18 e 19)

[15] MITRE Corporation, "CWE-770: Allocation of resources without limits or throttling," https://cwe.mitre.org/data/definitions/770.html, 2026, cWE versione 4.20. Consultato il 25 agosto 2026. [Online]. Available: https://cwe.mitre.org/data/definitions/770.html (Citato a pagina 19)

[16] M. Nottingham and R. T. Fielding, "Additional HTTP status codes," Internet Engineering Task Force, RFC 6585, apr 2012. (Citato a pagina 19)

[17] R. Polli, A. Martinez Ruiz, and D. Miller, "RateLimit header fields for HTTP," IETF HTTPAPI Working Group, Internet-Draft draft-ietf-httpapi-ratelimit-headers-11, may 2026, work in progress. https://datatracker.ietf.org/doc/draft-ietf-httpapi-ratelimit-headers/. (Citato a pagina 19)

[18] R. Chandramouli, "Security strategies for microservices-based application systems," National Institute of Standards and Technology, NIST Special Publication 800-204, aug 2019. (Citato a pagina 19)

[19] OWASP API Security Project, "API5:2023 broken function level authorization," https://owasp.org/API-Security/editions/2023/en/

0xa5-broken-function-level-authorization/, 2023, oWASP API Security Top 10 – 2023. Consultato il 25 agosto 2026. [Online]. Available: https://owasp.org/API-Security/editions/2023/en/0xa5-broken-function-level-authorization/ (Citato alle pagine 19, 20, 21 e 48)

[20] MITRE Corporation, "CWE-285: Improper Authorization," https://cwe.mitre.org/data/definitions/285.html, common Weakness Enumeration. [Online]. Available: https://cwe.mitre.org/data/definitions/285.html (Citato alle pagine 20 e 49)

[21] OWASP API Security Project, "API8:2023 security misconfiguration," https://owasp.org/API-Security/editions/2023/en/0xa8-security-misconfiguration/, 2023, oWASP API Security Top 10 – 2023. Consultato il 25 agosto 2026. [Online]. Available: https://owasp.org/API-Security/editions/2023/en/0xa8-security-misconfiguration/ (Citato alle pagine 21, 24, 25, 26 e 54)

[22] ——, "API9:2023 Improper Inventory Management," https://owasp.org/API-Security/editions/2023/en/0xa9-improper-inventory-management/, 2023, consultato il 25 agosto 2026. [Online]. Available: https://owasp.org/API-Security/editions/2023/en/0xa9-improper-inventory-management/ (Citato alle pagine 21, 26 e 27)

[23] ——, "API6:2023 Unrestricted Access to Sensitive Business Flows," https://owasp.org/API-Security/editions/2023/en/0xa6-unrestricted-access-to-sensitive-business-flows/, 2023, consultato il 25 agosto 2026. [Online]. Available: https://owasp.org/API-Security/editions/2023/en/0xa6-unrestricted-access-to-sensitive-business-flows/ (Citato alle pagine 22, 23, 31 e 65)

[24] ——, "API7:2023 server side request forgery," https://owasp.org/API-Security/editions/2023/en/0xa7-server-side-request-forgery/, 2023, oWASP API Security Top 10 – 2023. Consultato il 25 agosto 2026. [Online]. Available: https://owasp.org/API-Security/editions/2023/en/0xa7-server-side-request-forgery/ (Citato alle pagine 23, 24 e 52)

[25] MITRE Corporation, "CWE-918: Server-side request forgery (SSRF)," https://cwe.mitre.org/data/definitions/918.html, 2026, cWE versione 4.20. Consultato il 25 agosto 2026. [Online]. Available: https://cwe.mitre.org/data/definitions/918.html (Citato alle pagine 24 e 52)

[26] ——, "CWE-16: Configuration," https://cwe.mitre.org/data/definitions/16.html, 2026, categoria CWE, versione 4.20; mappatura vietata. Consultato il 25 agosto 2026. [Online]. Available: https://cwe.mitre.org/data/definitions/16.html (Citato a pagina 25)

[27] ——, "CWE-1059: Insufficient technical documentation," https://cwe.mitre.org/data/definitions/1059.html, 2026, cWE versione 4.20; denominazione precedente *Incomplete Documentation* fino ad aprile 2022; mappatura vietata. Consultato il 25 agosto 2026. [Online]. Available: https://cwe.mitre.org/data/definitions/1059.html (Citato a pagina 26)

[28] OWASP API Security Project, "Release notes – OWASP API security top 10 2023," https://owasp.org/API-Security/editions/2023/en/0x04-release-notes/, 2023, consultato il 25 agosto 2026. [Online]. Available: https://owasp.org/API-Security/editions/2023/en/0x04-release-notes/ (Citato a pagina 28)

[29] ——, "API10:2023 Unsafe Consumption of APIs," https://owasp.org/API-Security/editions/2023/en/0xaa-unsafe-consumption-of-apis/, 2023, consultato il 25 agosto 2026. [Online]. Available: https://owasp.org/API-Security/editions/2023/en/0xaa-unsafe-consumption-of-apis/ (Citato alle pagine 28, 29 e 55)

[30] V. Felmetsger, L. Cavedon, C. Kruegel, and G. Vigna, "Toward automated detection of logic vulnerabilities in web applications," in *Proceedings of the 19th USENIX Security Symposium (USENIX Security 10)*. Washington, DC: USENIX Association, 2010, pp. 143–160. (Citato a pagina 31)

[31] G. Pellegrino and D. Balzarotti, "Toward black-box detection of logic flaws in web applications," in *Proceedings of the 21st Network and Distributed System Security Symposium (NDSS 2014)*, San Diego, CA, 2014. (Citato a pagina 31)

[32] Prisma Cloud, "Checkov – what is checkov?" https://www.checkov.io/, 2026, analizzatore statico per Infrastructure-as-Code. [Online]. Available: https://www.checkov.io/ (Citato a pagina 35)

[33] HashiCorp, "Terraform documentation," https://developer.hashicorp.com/terraform/docs, 2026. [Online]. Available: https://developer.hashicorp.com/terraform/docs (Citato a pagina 35)

[34] M. Brunsfeld *et al.*, "Tree-sitter: an incremental parsing system for programming tools," https://tree-sitter.github.io/tree-sitter/, 2026. [Online]. Available: https://tree-sitter.github.io/tree-sitter/ (Citato a pagina 39)

[35] Stoplight, Inc., "Spectral – api style guide linter," https://docs.stoplight.io/docs/spectral, 2026, linter per specifiche OpenAPI e AsyncAPI. [Online]. Available: https://docs.stoplight.io/docs/spectral (Citato a pagina 42)

[36] ZAP Dev Team, "ZAP – zed attack proxy documentation," https://www.zaproxy.org/docs/, 2026. [Online]. Available: https://www.zaproxy.org/docs/ (Citato a pagina 42)

[37] J. Yang, E. Wang, J. Chen, Q. Wang, Y. Zhang, H. Duan, W. Xie, and B. Wang, "Token Time Bomb: Evaluating JWT Implementations for Vulnerability Discovery," in *Proceedings of the 32nd Network and Distributed System Security Symposium (NDSS)*. Internet Society, 2025. (Citato a pagina 43)

[38] B. Xu, S. Jia, J. Lin, F. Zheng, Y. Ma, L. Liu, X. Gu, and L. Song, "JWTKey: Automatic Cryptographic Vulnerability Detection in JWT Applications," in *Computer Security – ESORICS 2023*, ser. Lecture Notes in Computer Science, vol. 14346. Cham: Springer, 2023, pp. 263–282. (Citato a pagina 43)

[39] OWASP API Security Project, "API3:2023 Broken Object Property Level Authorization," https://owasp.org/API-Security/editions/2023/en/0xa3-broken-object-property-level-authorization/, 2023, consultato il 25 agosto 2026. [Online]. Available: https://owasp.org/API-Security/editions/2023/en/0xa3-broken-object-property-level-authorization/ (Citato a pagina 46)

[40] MITRE Corporation, "CWE-862: Missing Authorization," https://cwe.mitre.org/data/definitions/862.html, common Weakness Enumeration. [Online]. Available: https://cwe.mitre.org/data/definitions/862.html (Citato a pagina 49)

[41] ——, "CWE-650: Trusting HTTP Permission Methods on the Server Side," https://cwe.mitre.org/data/definitions/650.html, common Weakness Enumeration. [Online]. Available: https://cwe.mitre.org/data/definitions/650.html (Citato a pagina 49)

[42] ——, "CWE-284: Improper Access Control," https://cwe.mitre.org/data/definitions/284.html, common Weakness Enumeration. [Online]. Available: https://cwe.mitre.org/data/definitions/284.html (Citato a pagina 49)

[43] ——, "CWE-276: Incorrect Default Permissions," https://cwe.mitre.org/data/definitions/276.html, common Weakness Enumeration. [Online]. Available: https://cwe.mitre.org/data/definitions/276.html (Citato a pagina 49)

[44] ——, "CWE-269: Improper Privilege Management," https://cwe.mitre.org/data/definitions/269.html, common Weakness Enumeration. [Online]. Available: https://cwe.mitre.org/data/definitions/269.html (Citato a pagina 49)

[45] M. Wessels, S. Koch, G. Pellegrino, and M. Johns, "SSRF vs. developers: A study of SSRF-defenses in PHP applications," in *Proceedings of the 33rd USENIX Security Symposium (USENIX Security 24)*. Philadelphia, PA: USENIX Association, 2024, pp. 6777–6794. (Citato a pagina 53)

[46] Y. Ji, T. Dai, Z. Zhou, Y. Tang, and J. He, "Artemis: Toward accurate detection of server-side request forgeries through LLM-assisted inter-procedural path-sensitive taint analysis," *Proceedings of the ACM on Programming Languages*, 2025, preprint disponibile come arXiv:2502.21026. (Citato a pagina 53)

[47] E. Wang *et al.*, "Where URLs become weapons: Automated discovery of SSRF vulnerabilities in web applications," in *Proceedings of the 45th IEEE Symposium on Security and Privacy (S&P 2024)*. IEEE, 2024. (Citato a pagina 53)

[48] OWASP API Security Project, "API10:2023 unsafe consumption of APIs – conflicting vulnerability indicators," https://github.com/OWASP/API-Security/issues/80, 2023, issue n. 80 del repository GitHub OWASP/API-Security, marzo 2023. Consultato il 25 agosto 2026. [Online]. Available: https://github.com/OWASP/API-Security/issues/80 (Citato a pagina 56)

[49] MITRE Corporation, "CWE-20: Improper Input Validation," https://cwe.mitre.org/data/definitions/20.html, common Weakness Enumeration. [Online]. Available: https://cwe.mitre.org/data/definitions/20.html (Citato a pagina 56)

[50] ——, "CWE-319: Cleartext Transmission of Sensitive Information," https://cwe.mitre.org/data/definitions/319.html, common Weakness Enumeration. [Online]. Available: https://cwe.mitre.org/data/definitions/319.html (Citato a pagina 56)

[51] ——, "CWE-601: URL Redirection to Untrusted Site (Open Redirect)," https://cwe.mitre.org/data/definitions/601.html, common Weakness Enumeration. [Online]. Available: https://cwe.mitre.org/data/definitions/601.html (Citato a pagina 56)

[52] OWASP Foundation, "crAPI: completely ridiculous API," https://github.com/OWASP/crAPI, 2026. [Online]. Available: https://github.com/OWASP/crAPI (Citato a pagina 59)

[53] NCC Group, "sadcloud: a tool for standing up insecure AWS infrastructure," https://github.com/nccgroup/sadcloud, 2021, revisione f538652 del 9 luglio 2021. [Online]. Available: https://github.com/nccgroup/sadcloud (Citato a pagina 60)

[54] erev0s, "VAmPI: vulnerable API made with Flask," https://github.com/erev0s/VAmPI, 2026, revisione f16052d del 7 aprile 2026. [Online]. Available: https://github.com/erev0s/VAmPI (Citato a pagina 60)

[55] Payatu, "DVAPI: damn vulnerable API," https://github.com/payatu/DVAPI, 2025, revisione bde30d2 del 22 settembre 2025. [Online]. Available: https://github.com/payatu/DVAPI (Citato a pagina 60)

[56] F. Sun, L. Xu, and Z. Su, "Static detection of access control vulnerabilities in web applications," in *Proceedings of the 20th USENIX Security Symposium (USENIX Security 11)*. San Francisco, CA: USENIX Association, 2011. (Citato a pagina 64)

[57] S. Son, K. S. McKinley, and V. Shmatikov, "RoleCast: Finding missing security checks when you do not know what checks are," in *Proceedings of the 2011 ACM International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA 2011)*. ACM, 2011, pp. 1069–1084. (Citato a pagina 64)

[58] M. Monshizadeh, P. Naldurg, and V. N. Venkatakrishnan, "MACE: Detecting privilege escalation vulnerabilities in web applications," in *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security (CCS 2014)*. ACM, 2014, pp. 690–701. (Citato a pagina 64)

[59] V. Atlidakis, P. Godefroid, and M. Polishchuk, "Checking security properties of cloud service REST APIs," in *Proceedings of the 13th IEEE International Conference on Software Testing, Validation and Verification (ICST 2020)*. IEEE, 2020, pp. 387–397. (Citato a pagina 64)

[60] J. Antunes, N. F. Neves, and P. Veríssimo, "Detection and prediction of resource-exhaustion vulnerabilities," in *Proceedings of the 19th IEEE International Symposium on Software Reliability Engineering (ISSRE 2008)*. IEEE, 2008. (Citato a pagina 64)

Ringraziamenti

Ringraziamenti qui...